

Article

An Automated Evolutionary Modularization Approach for Performance-Aware Parallelization of Sequential Source Code

Kamran Adil Ibrahim, Habib Izadkhah *  and Jaber Karimpour

Department of Computer Science, Faculty of Mathematics, Statistics, and Computer Science, University of Tabriz, Tabriz 5166616471, Iran

* Correspondence: izadkhah@tabrizu.ac.ir

Abstract

Sequential object-oriented software remains difficult to parallelize because several important decisions are not entirely local. A useful decomposition should consider architectural structure, execution frequency, communication latency, and how much communication can overlap with local computation. In contrast, many traditional modularization methods focus on cohesion, coupling, or communication cost without explicitly considering whether an unavoidable remote call can be issued early enough for part of its latency to be hidden by independent work. DTMC-GA addresses this problem by combining a Discrete-Time Markov Chain (DTMC) execution model, a Distributive Execution Time (DET) estimator, and a Genetic Algorithm (GA) search procedure. Guarded call-use displacement enlarges modeled overlap windows only when the relevant dependence, ordering, and safety conditions are satisfied. The public artifact contains five local benchmark fixtures, five comparison methods, 30 retained runs per algorithm–fixture pair, raw outputs, statistical scripts, regenerated figures, Docker support, continuous-integration verification, and a Zenodo DOI. Within this released five-method panel, DTMC-GA achieved the highest aggregate mean modeled speedup of 2.708 ± 0.329 . A more recent NSGA-III comparator was also evaluated using matched fixtures, DTMC–DET inputs, constraints, random seeds, population size, and generation budget. NSGA-III achieved an aggregate mean modeled speedup of 2.737 ± 0.323 , 1.07% above DTMC-GA, although none of the five fixture-level paired differences remained significant after Bonferroni correction. A separate sparse-graph scalability study covered 50, 100, 250, 500, 750, and 1000 classes; mean analysis time increased from about 0.024 to 0.286 s, while peak memory rose from about 119.1 to 120.9 MiB. The study also includes an executed OpenCV 4.7.0 calibration using a fixed five-convolution DNN workload. Across 30 matched pairs, native-parallel OpenCV achieved a measured ratio-of-means speedup of $1.8691 \times$. The bounded empirical DTMC-GA/DET rerun over 30 seeds produced a mean modeled speedup of $1.0000 \times$ with an SD of 0.0000, giving an absolute modeled-versus-measured difference of 46.50%. Because the measured parallel comparator is OpenCV’s native threaded implementation rather than a DTMC-GA-generated source rewrite, this experiment is interpreted as real-runtime model calibration rather than direct measured DTMC-GA optimized speedup.



Academic Editors: Xueyi Li and Paolo Bellavista

Received: 14 July 2026

Revised: 31 August 2026

Accepted: 1 September 2026

Published: 10 September 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article

distributed under the terms and

conditions of the [Creative Commons](#)

[Attribution \(CC BY\)](#) license.

Keywords: automated software modularization; concurrency-aware modularization; parallelization; discrete-time Markov chain; genetic algorithm; distributive execution time; reproducibility artifact

1. Introduction

Modern software increasingly runs on multi-core, distributed, or service-oriented platforms, yet many object-oriented systems still begin as sequential codebases. In such systems, parallelization is not only a compiler-level transformation. It is also an architectural choice on whether classes should be grouped together, whether there should be interactions across module boundaries, and whether unavoidable communication can be overlapped with useful local work. Restructuring large legacy codebases is costly and fragile, which explains the long-standing interest in automated modularization. Most existing approaches consider the problem predominantly in terms of reducing inter-module communication cost (MCC). Although this is an important aspect, communication that cannot be eliminated may still be hidden through overlap with independent computation. The central question of this study is therefore how structural modularization can be coupled with a reproducible model of execution likelihood, overlap potential, and deployment constraints.

Such explicit restrictions on the set of admissible modularization solutions are called deployment constraints in this study. These restrictions can be the number and size of modules; class placement and co-location; resource-budget limits; and transformation-safety conditions. Network latency and bandwidth are represented by the communication-cost part of the model if they are not hard-constrained. As a result, the optimization procedure works in a deployment space that considers both structural needs and implementation constraints.

Typically, static modularization methods [1–6], such as the Bunch algorithm and the fast clustering algorithm FCA [7], begin by considering a Class Dependence Graph (CDG) and look for partitions that have desirable cohesion and coupling. Bunch is based on a genetic algorithm and a modularization quality (MQ) measure, which is based on intra and inter module edges, FCA is based on a heuristic near-linear hierarchical clustering. The approaches are useful in recovering structural partitions but the recovered structure alone does not provide any information on the frequency of execution, probabilistic control-flow, and communication-computation overlap. The speedup that is expected must thus be explicitly modeled or measured if it is to be attributed to speedup.

Only recently, the exploration of the partition space has been improved by means of metaheuristic studies, but these are still oriented towards a structural goal; Schnoor et al. [8] also applied a genetic-algorithm-based approach to research-software structure optimization. Later, Arasteh et al. [9,10] introduced discretized sand cat and bedbug optimization approaches for remodularization and architectural-model generation from source code. Alshareef & Maashi [11] used a multi-objective hyper-heuristic for module clustering, and Prajapati & Geem [12] applied Harmony Search to multi-objective software architecture reconstruction. These methods expand the search method and their fitness functions can still be based on structural quality instead of an estimated time to execute the structure in parallel.

Compiler-based parallelization addresses a different level of granularity. Tools such as Cetus [13] perform source-to-source transformations and can insert OpenMP directives for loop-level parallelism. Polaris [14], SUIF [15], and Polly [16] similarly focus on loops, arrays, and fine-grained transformations, often through dependence or polyhedral analysis. These compilers can be highly effective inside methods, but they are not designed to decide how a large object-oriented system should be split into asynchronously communicating deployment modules. The higher-level architectural question—which classes should be co-located and which can be separated—is largely left to the developer.

Distributed programming models and object-distribution systems, including Charm++ (v.8.0.0) [17–20], Legion (v.26.06.0) [21], and PaRSEC (v.4.0.2411) [22], provide abstractions such as migratable objects, logical regions, and task graphs. They help programmers express concurrency once a decomposition is known. They do not, however,

infer that decomposition from a sequential object-oriented codebase. A similar limitation applies to CUDA [23] and OpenMP [24]: both are useful when parallel regions or kernels have already been identified, but neither solves the upstream architectural modularization problem.

Probabilistic state-based models offer a complementary perspective. Li et al. [25] used DTMCs to predict runtime reliability in self-adaptive systems by representing dynamic execution paths and failure probabilities. Cheung [26] developed a user-oriented Markov reliability model, Gokhale et al. [27] examined architecture-based software performance and reliability prediction, and Souravlas et al. [28] used Markov-process modeling for dynamic load balancing on heterogeneous cloud platforms. The studies demonstrate that information about the dynamics of execution can be encoded within DTMCs, but the Markov model is not used to determine which modules to select for architecture-level boundaries, it is used for reliability or scheduling instead.

Another strand that is relevant to search-based software engineering (SBSE). The early framing of search-based software engineering was established by Harman and Jones [29], while Harman et al. [30] later reviewed its trends, techniques, and applications. Praditwong et al. [31] formulated software module clustering as a multi-objective search problem; NSGA-II [32] is a widely used Pareto-based evolutionary algorithm relevant to this class of optimization. Once again, the fitness functions are structural and the Pareto fronts display trade-offs between cohesion/coupling and not between communication/speedup. More recent software-clustering studies have explored explainability and machine-learning-assisted customization: Tan et al. [33] proposed E-SC4R for explaining software clustering in remodularization, while Meng et al. [34] proposed CLUE to customize clustering techniques using machine learning. These approaches improve the analysis and selection of software clusters but do not provide the DTMC-DET performance model used here for parallel-deployment evaluation.

The analytical approach closest to the Distributability Verification Model of Isazadeh et al. [35] introduced the Distributive Execution Time (DET) concept used as the basis for the estimator in this work. For a fixed assignment of synchronous and asynchronous calls, DET can determine whether a selected partition reduces overall execution time. Its role is therefore evaluative rather than generative: it assumes that a modularization already exists, does not search alternative partitions, and does not attempt to create additional overlap by safely moving call and use instructions apart.

The related work therefore provides several strong but separate ingredients: MQ-based structural modularization [1,5,7], metaheuristic remodularization [9,11,12], compiler and programming-framework support for already identified parallelism [14–17], DTMC-based reliability or scheduling models [25,28], and DET-based verification [35]. The gap is not a lack of individual techniques. It is the lack of a reproducible framework that connects execution likelihood, speedup-aware DET evaluation, evolutionary partition search, guarded code-motion opportunities, and deployment constraints in one bounded pipeline.

Table 1 synthesizes this positioning and shows the dimensions on which DTMC-GA differs from prior approaches.

To address this gap, DTMC-GA shifts the optimization target from communication minimization alone to concurrency-aware modularization. The method combines three elements in a single search pipeline: (1) a Discrete-Time Markov Chain (DTMC) that assigns probabilistic weight to execution paths and expected visits; (2) an extended Distributive Execution Time (DET) view that estimates completion time under synchronous and asynchronous call assignments while accounting for latency and overlap; and (3) a Genetic Algorithm (GA) that searches feasible class partitions with a speedup-aware fitness function and explicit deployment penalties.

Table 1. Methodological positioning of modularization and parallelization approaches.

Approach	Technique	Modeling	Concurrency View	Objective	Limits/Constraints	Code/Transformation	Scale
Bunch [1]	GA clustering	Static CDG	None (MQ)	Single-objective GA	Limited	No	Up to 500
FCA [7]	Hierarchical clustering	Static CDG	None (MQ)	Greedy heuristic	No	No	Up to 1000
Sand Cat [9]	Swarm optimization	Static CDG	None (MQ)	Metaheuristic	No	No	Standard benchmarks
Bedbug [10]	Swarm optimization	Static CDG	None (MQ)	Metaheuristic	No	No	Standard benchmarks
Hyper-heuristic [11]	Hyper-heuristic	Static CDG	None (MQ)	Multi-objective hyper-heuristic	No	No	Benchmarks
Harmony Search [12]	Multi-objective HS	Static CDG	None (MQ)	Multi-objective metaheuristic	No	No	Benchmarks
Isazadeh et al. [35]	Analytical DET	Deterministic	DET (fixed)	None (verification)	Yes	No	Small (<100)
Cetus [13]	Source-to-source	Static (loops)	None (loop speedup)	Heuristic passes	No	Loop parallelization	N/A (loops)
DTMC reliability [25]	DTMC prediction	Probabilistic	None (reliability)	None	No	No	Small
E-SC4R [33]	Explainable software clustering	Clustering-based modularization	None	Clustering interpretation	No	No	Moderate
SBSE clustering [29,31]	GA/NSGA-II	Static CDG	None (MQ)	Multi-objective evolutionary	No	No	Moderate
DTMC-GA (this work)	DTMC + GA + DET	Probabilistic DTMC	Speedup-based MQ	Constrained GA	Yes (penalties)	Call-use displacement	5 local fixtures

The methodology also incorporates a call-use displacement transformation, which is similar to code motion optimizations performed by the parallelizing compilers, to try to increase the distance between the first use of a result from a remote method call and the call itself. Depending on the safety conditions, the transformation may provide other concurrency opportunities that the DET model can take advantage of, thereby increasing the amount of overlap between communication and local computation in the modeled program, but without changing its semantics.

The motivation above is addressed by the following research questions, which tie into the formal model, implementation pipeline and fixture-suite evaluation:

1. **RQ1:** Can DTMC-based execution probabilities expose execution-relevance patterns for distributability that static unweighted call graphs do not capture?
2. **RQ2:** How can a Genetic Algorithm be combined with the DET model to search for modularizations that favor modeled concurrency rather than merely reduce communication?
3. **RQ3:** How does guarded call-use displacement affect modeled overlap and fixture-suite efficiency indicators, and how can this effect be quantified?
4. **RQ4:** How effectively do the proposed speedup-based metrics (DET and MQ) explain the artifact-generated parallel-performance indicators compared with traditional cohesion/coupling measures?
5. **RQ5:** How does DTMC-GA compare with communication-cost baselines on the released fixtures, and what further evidence is required for large-project generalization?

The resulting contributions are deliberately limited in scope. The study treats modularization as a performance-oriented search problem in which execution probabilities, overlap potential, communication cost, load balance, and deployment constraints are considered together rather than only cohesion and coupling. DTMC-derived expected visits are also incorporated into the DET formulation, so frequently executed collaborations have more influence than infrequent ones. In addition, the methodology uses conservative conditions for call-use displacement to reject unsafe transformations instead of assuming that every

call can be moved. Finally, the study provides a reproducibility artifact containing retained raw runs, analysis scripts, regenerated figures, Docker and continuous-integration configuration, and archival metadata, allowing the fixture-suite evidence to be independently inspected and repeated.

The evaluation is based mainly on the released artifact, which contains five local CSV fixtures, 30 retained seeds for each algorithm–fixture combination, raw and processed outputs, statistical scripts, regenerated figures, Docker configuration, continuous-integration verification, and a Zenodo DOI. It also includes a bounded real-project checkpoint using OpenCV 4.7.0 at source commit 725e440d278aca07d35a5e8963ef990572b07316. This checkpoint covers exact-source build verification, executed instrumentation, runtime measurement, and generation of a DTMC-GA/DET calibration from the collected traces. The additional experiment strengthens the connection between the proposed method and production software, but it is not a direct measurement of runtime performance from a DTMC-GA-generated OpenCV rewrite. The OpenCV evidence is therefore treated as a calibration and feasibility checkpoint rather than full validation of production-scale source transformation.

This paper continues with the following organization. The definition of the problem and optimization goal is made more explicit in Section 2. In Section 3, we describe the methodology we propose, which involves the construction of the call-dependency representation, modeling in a DTMC, performance estimation using DET, guarded call-use displacement, and GA-based modularization. The experimental design and reproducibility protocol are described in Section 4. The results of the experiments are summarized in Section 5, and the main findings, evidence boundaries, and remaining limitations are discussed in Section 6. Finally, Section 7 summarizes the study and offers suggestions for future research.

2. Problem Definition and Optimization Objective

The motivating problem is formalized here as the search objective used by DTMC-GA. Presenting the objective before the implementation details makes the notation, constraints, and optimization target explicit. The input is an object-oriented software system represented by a class set $V = \{C_1, C_2, \dots, C_n\}$, a directed set of call or dependence edges E , and a weight function $w(C_i, C_j)$ that records either statically estimated or dynamically observed invocation frequency. These elements define the weighted Class Dependence Graph (CDG), $G = (V, E, w)$.

A candidate modularization is a partition $S = \{M_1, M_2, \dots, M_k\}$ of V , where each class belongs to exactly one module and k is bounded by a user-specified maximum number of deployable modules. For any two classes C_i and C_j , an edge is intra-module if both classes are assigned to the same module and inter-module otherwise. Inter-module edges can be used to support asynchronous execution, but can also have a communication overhead.

The outgoing call frequencies of the CDG are used to obtain the DTMC transition matrix $\mathbf{P}$. Its entry, P_{ij} , represents the transition probability from class C_i to class C_j , calculated as the ratio of the number of calls from C_i to C_j to the total number of outgoing calls from C_i . Classes with no outgoing calls are modeled as absorbing classes. The expected visit counts calculated from the DTMC can then be used to identify execution hot spots and weight the performance contribution of the classes.

In order to be consistent with the notation used throughout the paper, S denotes a candidate modularization, Ω is the feasible set after module-count and deployment constraints are applied, and $f_{\text{exec}}(S)$ is the scalar fitness value optimized by the GA. The objective seeks a performance-aware modularization while controlling communication overhead, preserving load balance, and satisfying deployment requirements. The formal objective is therefore:

$$\begin{aligned}
 S^* &= \arg \max_{S \in \Omega} f_{\text{exec}}(S), \\
 f_{\text{exec}}(S) &= a_s \cdot S_{bDET}(S) + w_1 \cdot \text{MQ}_{\text{speedup}}(S) - w_2 \cdot \text{CommCost}(S) \\
 &\quad + w_3 \cdot \text{LoadBalance}(S) - w_4 \cdot \text{Penalty}(S).
 \end{aligned} \tag{1}$$

In Equation (1), the modeled-speedup ratio $S_{bDET}(S)$ is estimated by the DET evaluator, while $\text{MQ}_{\text{speedup}}(S)$ is the speedup-aware modularization term defined in the methodology. $\text{CommCost}(S)$ represents the overhead introduced by module boundaries, $\text{LoadBalance}(S)$ describes how the estimated execution time is distributed among modules, and $\text{Penalty}(S)$ represents deployment penalties when they are enabled. The overall formulation is therefore not merely a structural clustering criterion, but a constrained objective for performance optimization. Since the solver is metaheuristic, it does not claim that the feasible partition it finds is the global optimum; instead, it searches for a high-quality feasible solution.

For the retained executable fixture suite, the optimizer uses $a_s = 1.00$, $w_1 = 0.12$, $w_2 = 0.80$, $w_3 = 0.35$, and $w_4 = 0.00$. These coefficients are fixed across all five fixtures and all retained random seeds. They are used directly rather than normalized to sum to one because the objective components operate on different numerical scales. The coefficient a_s provides the reference scaling for the DET-based modeled speedup. The zero value assigned to w_4 reflects the retained implementation, in which the module cap and transformation-safety requirements are enforced as hard feasibility and safety conditions rather than as soft penalties.

In the general objective formulation, c refers to the number of configured penalties, and r refers to a particular penalty in the set. This notation is kept to allow for configurations where other deployment-specific penalties can be added. In the experimental fixture suite, however, the penalty coefficient is set to zero ($w_4 = 0.00$), as the corresponding feasibility and transformation-safety requirements are checked directly as hard constraints.

3. The Method

The methodology is based on the idea that communication cost alone is not enough to determine whether a modularization is suitable for parallel execution. DTMC-GA combines three components: a Discrete-Time Markov Chain (DTMC) for representing probabilistic execution behavior, a Distributive Execution Time (DET) model for estimating parallel completion time, and a Genetic Algorithm (GA) for searching for modularizations that improve modeled speedup while satisfying deployment constraints. The computational process is organized so that weighted class-dependency inputs can be traced through DTMC estimation, DET scoring, guarded-overlap updates, GA ranking, and the resulting artifact outputs. The workflow is divided into four stages: source analysis and construction of the class-dependence graph, DTMC modeling and structural analysis, distributability enhancement through guarded call-use displacement, and GA-based modularization.

The uppercase calligraphic symbols represent sets (e.g., $\mathcal{CC}(C_i)$ for collaborators and $\mathcal{U}$ for call-use pairs), bold uppercase symbols represent matrices (e.g., $\mathbf{P}$ and $\mathbf{F}$), and S denotes a candidate modularization. The notation P_{ij} and F_{ij} denote entries of $\mathbf{P}$ and $\mathbf{F}$, respectively, whereas $\mathcal{P}$ denotes the GA population.

This work starts with a precise description of the interaction between classes. Suppose C_k is a class in an object-oriented program. The set of collaborating classes, $\mathcal{CC}(C_k)$, contains all classes that interact with C_k , for example through method invocations or data dependencies. The strength of the collaboration between C_k and C_i is measured by the invocation count $f(C_k, C_i)$, which is used to obtain DTMC transition probabilities. The pairwise frequencies are combined into a weighted directed Class Dependence Graph (CDG), $G = (V, E, w)$, where V is the class set, an edge $(C_i, C_j) \in E$ indicates a method call from

C_i to C_j , and $w(C_i, C_j)$ stores the corresponding call frequency. The CDG is used for the subsequent probabilistic and performance-driven analysis.

A software module M_i is then defined as a disjoint subset of V and represents one deployment unit in the parallel architecture. The modularization problem therefore has two main goals: partition the classes into modules that expose concurrency opportunities and keep inter-module communication low.

The focus of this performance estimator is the Distributive Execution Time (DET) described by Isazadeh et al. [35] and expanded on here to include probabilistic execution paths. For a class C_i with collaborator set $\mathcal{CC}(C_i)$, the DET recursively accumulates the execution times of its internal sequential blocks and the times of its collaborators, each of which may be invoked either sequentially or asynchronously, as formulated in Equation (2):

$$\text{DET}_{C_i} = \sum_{k=1}^p t_k + \sum_{C_j \in \mathcal{CC}(C_i)} \left[a_j \cdot \text{DET}_{C_j} \right] + \sum_{C_j \in \mathcal{CC}(C_i)} \left[(1 - a_j) \cdot \max(\text{DET}_{C_j} - t_{\text{gap}} + 2C_{ij} + I_{\text{init}}, 0) \right]. \quad (2)$$

where t_k are the local sequential block times, $a_j \in \{0, 1\}$ indicates whether the call to C_j is sequential ($a_j = 1$) or asynchronous ($a_j = 0$), I_{init} is the remote invocation overhead, C_{ij} the communication time between the two classes, and t_{gap} the window between issuing the call and the first use of its result during which the caller can perform useful work. When $a_j = 1$, the caller blocks until the callee completes; when $a_j = 0$, the remote execution overlaps with local processing, reducing the net addition to the caller's time to the positive part of $\text{DET}_{C_j} - t_{\text{gap}} + 2C_{ij} + I_{\text{init}}$.

Because the execution mode vector $\mathbf{A} = [a_1, \dots, a_m]$ must be chosen for every collaborative relationship, the modeled best execution strategy can be expressed as the discrete optimization problem shown in Equation (3):

$$\mathbf{A}^* = \arg \min_{\mathbf{A}} \sum_{i=1}^n \text{DET}_{C_i}(\mathbf{A}), \quad (3)$$

where n is the number of classes and m is the total number of edges in the CDG. The recursive nature of DET and the binary mode choices make this a challenging combinatorial problem, which is addressed with the genetic algorithm described later.

Example 1 (Applying DET to a multiclass system). Consider the set of classes $\{A, \dots, H\}$ shown in Figure 1. In the general DET notation, t_k denotes the execution time of the k -th local sequential code block. In this running example, the same block-level timing is written as $c_{X,k}$ when the block belongs to class X ; thus, X identifies the class and k identifies the block within that class. These execution times can be obtained by profiling the application on the target machine or, when appropriate, by using a WCET estimation technique. The notation $c_{X,k}$ is used both to construct the local sequential execution time of each class and to define the overlap window t_{gap} for the asynchronous calls listed in Table 2.

The Distributive Execution Time (DET) for a class C_i follows Equation (4):

$$\text{DET}_{C_i} = \sum_k t_k + \sum_{C_j \in \mathcal{CC}(C_i)} \left[a_j \text{DET}_{C_j} + (1 - a_j) \max(\text{DET}_{C_j} - t_{\text{gap}} + 2C_{ij} + I_{\text{init}}, 0) \right], \quad (4)$$

where $a_j \in \{0, 1\}$ determines whether the interaction with collaborator C_j is sequential or asynchronous, I_{init} is the invocation overhead, C_{ij} is the communication cost between classes C_i and C_j , and t_{gap} is the interval between issuing an asynchronous call and the first use of its result,

during which useful local computation may overlap with remote execution. The local sequential execution times are $T_A^{loc} = \sum_{k=0}^6 c_{A,k}$, $T_B^{loc} = c_{B,7} + 5(c_{B,10} + c_{B,11})$, $T_C^{loc} = \sum_{k=12}^{14} c_{C,k}$, $T_D^{loc} = c_{D,15} + 5c_{D,16} + c_{D,17}$, $T_E^{loc} = \sum_{k=18}^{20} c_{E,k}$, $T_F^{loc} = c_{F,21}$, $T_G^{loc} = \sum_{k=22}^{23} c_{G,k}$, and $T_H^{loc} = c_{H,24}$. For example, $c_{A,3}$ is the execution time of block t_3 in class A and is used as t_{gap} for the asynchronous edge $A \rightarrow E$; Table 2 similarly uses $c_{A,5}$, $c_{D,16}$, and $c_{E,19}$ for the other asynchronous overlap windows.

The four asynchronous edges ($A \rightarrow E$, $A \rightarrow F$, $D \rightarrow C$, $E \rightarrow H$) allow execution-mode switching through the vector $\mathbf{A} = (a_{AE}, a_{AF}, a_{DC}, a_{EH})$. Different execution-mode combinations are therefore evaluated to identify the configuration with the minimum total system DET, as defined in Equation (5):

$$\mathbf{A}^* = \arg \min_{\mathbf{A} \in \{0,1\}^4} \sum_{C_i \in \{A, \dots, H\}} \text{DET}_{C_i}(\mathbf{A}). \quad (5)$$

Because the DET calculation contains recursive class dependencies and max operators, the resulting combinatorial optimization problem is solved using the proposed Genetic Algorithm.

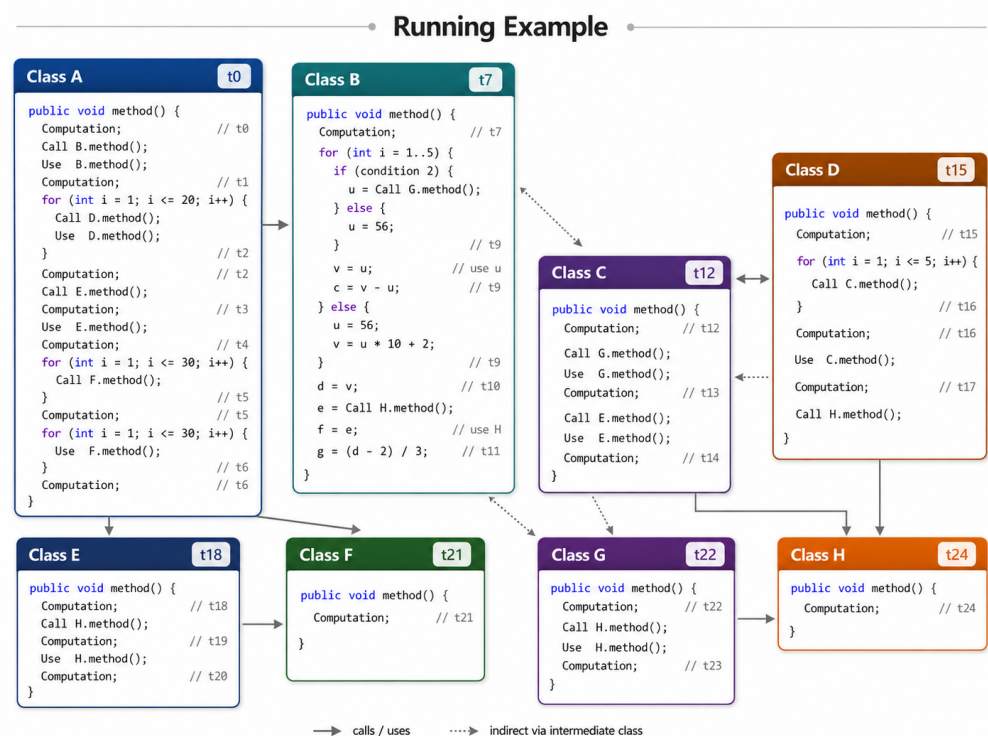


Figure 1. Eight-class running example with timing markers (t_0 – t_{24}) and class-interaction cues. Colors distinguish the class boxes only and carry no quantitative meaning; solid arrows indicate direct call/use relationships, whereas dotted arrows indicate indirect interaction via an intermediate class.

Table 2. Call characteristics for the example system.

Caller	Callee	Count	Mode (a_j)	t_{gap}	Caller	Callee	Count	Mode (a_j)	t_{gap}
A	B	1	Seq (1)	–	C	G	1	Seq (1)	–
A	D	20	Seq (1)	–	C	E	1	Seq (1)	–
A	E	1	Async (0)	$c_{A,3}$	D	C	5	Async (0)	$c_{D,16}$
A	F	30	Async (0)	$c_{A,5}$	D	H	1	Seq (1)	–
B	G	≤ 5	Seq (1)	–	E	H	1	Async (0)	$c_{E,19}$
B	H	5	Seq (1)	–	G	H	1	Seq (1)	–

3.1. Integrated Methodology Pipeline

Figure 2 summarizes the four-stage path from source-code structure to a performance-aware modularization. Each stage produces information used by the next stage, which keeps the pipeline auditable from CDG construction through DTMC analysis, DET evaluation, guarded displacement, and GA search.

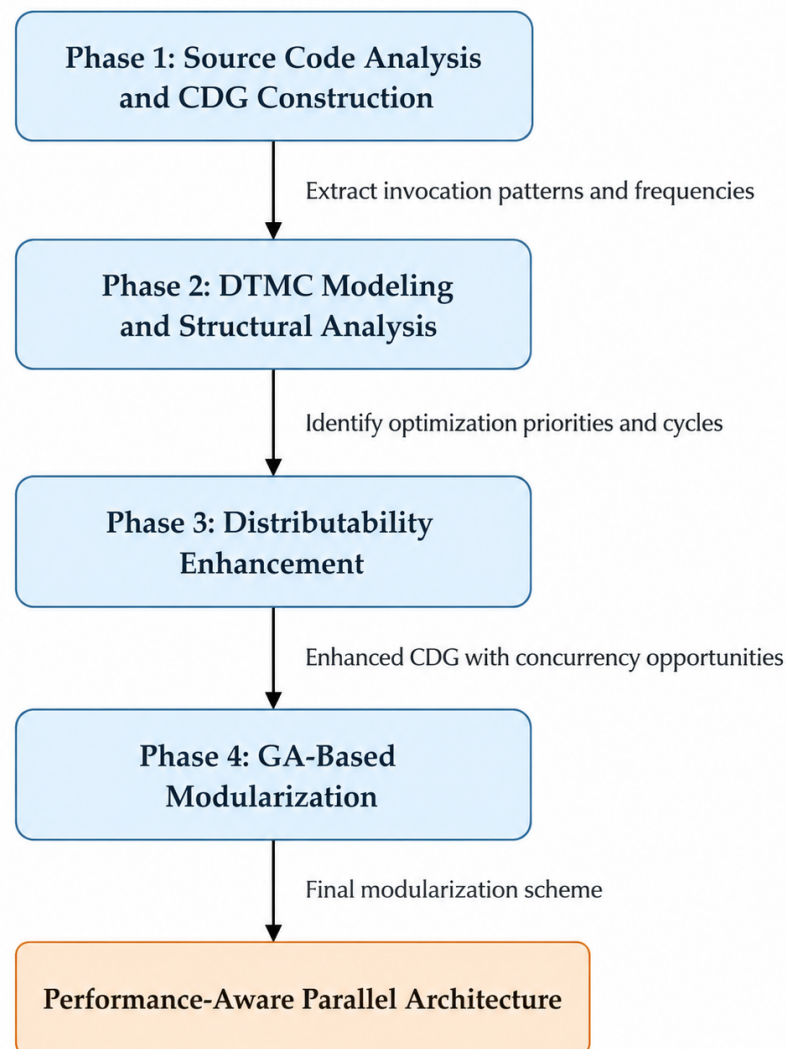


Figure 2. Four-phase DTMC-GA methodology overview. Blue boxes denote the four processing phases, the orange box denotes the resulting performance-aware parallel architecture, and arrows indicate the sequential flow of information.

3.2. Phase 1: Source-Code Analysis and CDG Construction

In the full source-analysis architecture, the pipeline starts by parsing C++ source code with the Clang/LLVM front-end [36], which produces an Abstract Syntax Tree (AST) for each translation unit. The analysis then extracts class declarations, inheritance hierarchies, method signatures, and method calls. Class hierarchy analysis [37] can be used to approximate virtual calls, and, when dynamic calls are not available, invocation frequencies can be estimated from static information like loop nesting depth and branch probabilities. Such data will fill the CDG; each node represents a class and the weight of each directed edge from C_i to C_j reflects the frequency of calls from C_i to C_j 's methods. The final graph shows the structure of the calls that the DTMC and DET stages use.

The methodology can adopt a hybrid Worst-Case Execution Time (WCET) strategy to provide the per-method execution times (t_k) for the input of the DET model. Short code segments can be profiled on the target machine and critical paths can be checked using a static WCET analyzer. There are several tools to aid this process, such as aiT [38], Bound-T [39], OTAWA [40], Chronos [41], and RapiTime [42]. The detailed pipeline of this stage from source code to validated CDG and per-method timings used for creating the DET model is illustrated in Figure 3.

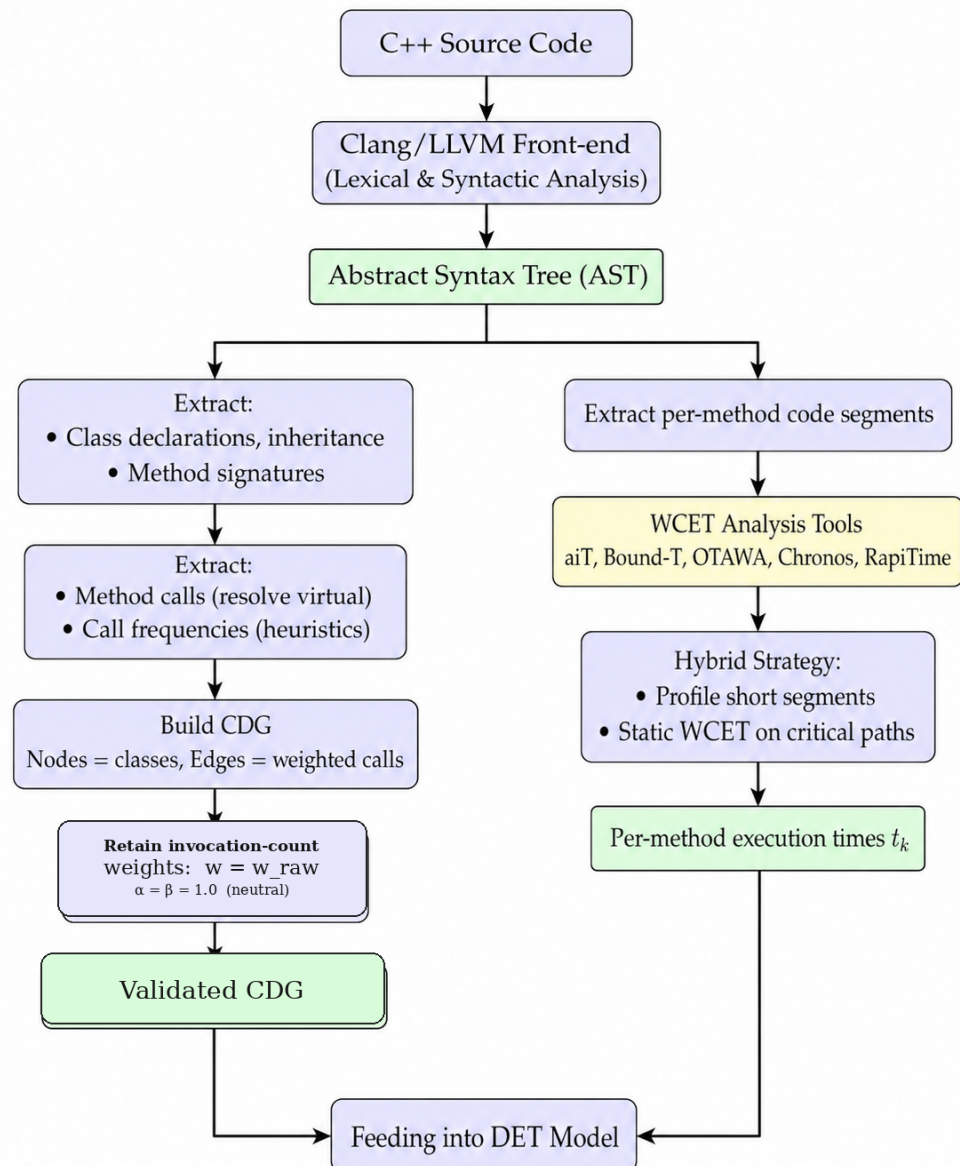


Figure 3. Phase 1 workflow from source code to validated CDG and method timings for DET evaluation. Purple boxes denote analysis/extraction steps, green boxes denote validated/intermediate outputs, the yellow box denotes WCET-analysis tools, and arrows show the data flow between stages.

The revised methodology explicitly addresses the inputs for the timing and communication aspects. In the released fixture suite, class-level and method-level timing fields and quantities related to communication are documented inputs included in the benchmark CSV artifacts used by the DET evaluator. The timing information and dynamic invocation counts for the OpenCV calibration come from the instrumented execution traces. Structural trace parents for which there are no metric-bearing timing records are retained with a

local time of 0.0 rather than being assigned an inferred execution time. Since the bounded OpenCV adapter is used in a single process, no inter-process communication volume was measured, so the cost of communication is set to 0.0. Likewise, there is no assumption of parallel overlap for the deliberately single-threaded profiling traces.

In the experiments reported in this study, the inheritance and dynamic-binding multipliers are kept at their neutral values, with $\alpha = \beta = 1.0$. Consequently, the multiplication shown in Algorithm 1 does not alter the numerical weight of any call edge, and the retained edge weight remains equal to the accumulated invocation count. Information on inheritance relationships and virtual-call resolution is still preserved as structural metadata, but it does not introduce any additional numerical amplification in the current evaluation. The more general formulation is retained to accommodate possible future configurations. If non-neutral multiplier values are introduced in future work, they will require independent calibration and sensitivity analysis before they can be treated as empirically supported parameters.

Algorithm 1 Source-code analysis and CDG construction in Phase 1

Require: C++ source files $\mathcal{F}$; target machine profiling environment

Ensure: Class Dependence Graph $G = (V, E, w)$; per-method WCET estimates $\{t_k\}$

```

1: // Step 1: Parse and build AST
2: for all file  $f \in \mathcal{F}$  do
3:   Parse  $f$  with Clang/LLVM front-end, obtain translation unit  $T_f$ 
4:   Extract AST  $\mathcal{T}_f$  from  $T_f$ 
5: end for
6: // Step 2: Structural extraction
7:  $V \leftarrow \emptyset$ ;  $E \leftarrow \emptyset$ ;  $w \leftarrow \emptyset$ 
8: for all AST node  $n \in \bigcup \mathcal{T}_f$  do
9:   if  $n$  is a class declaration then
10:    Add class  $C_n$  to  $V$ , store inheritance hierarchy
11:   else if  $n$  is a method signature then
12:    Record method  $m$  with its containing class  $C_m$ 
13:   else if  $n$  is a method call then
14:    Resolve virtual call via class hierarchy analysis
15:    Let caller class  $C_i$ , callee class  $C_j$ 
16:    Estimate invocation count  $cnt$  using static heuristics
17:    If  $(C_i, C_j) \notin E$ , create edge with weight 0
18:     $w(C_i, C_j) \leftarrow w(C_i, C_j) + cnt$ 
19:   end if
20: end for
21: // Step 3: Hybrid WCET timing
22: for all method  $m$  in each class do
23:   if  $m$  is a short, simple method then
24:    Profile  $m$  on target machine, obtain execution time  $t_m$ 
25:   else
26:    Apply static WCET analyzer (aiT, Bound-T, OTAWA, etc.)
27:    Obtain safe upper bound  $t_m$ 
28:   end if
29:   Store  $t_m$  as the sequential block time for the corresponding timing marker
30: end for
31: // Step 4: Build final CDG
32: for all edge  $(C_i, C_j) \in E$  do
33:   Optionally enhance weight:  $w(C_i, C_j) \leftarrow w(C_i, C_j) \cdot \alpha \cdot \beta$   $\triangleright \alpha$ : inheritance factor,  $\beta$ :
    dynamic binding factor
34: end for
35: return  $G = (V, E, w)$  and the set of per-method times  $\{t_k\}$ 

```

3.3. Phase 2: DTMC Modeling and Structural Analysis

The CDG provides a static view of dependency, but the execution isn't uniform: some classes are executed repeatedly, while other classes are executed on rare paths. Therefore, DTMC-GA transforms the CDG to a Discrete-Time Markov Chain (DTMC), following the general trend of employing DTMCs in software reliability prediction models by Li et al. [25]. Each class is represented as a state, each transition corresponds to a method call, and the transition probability from C_i to C_j is derived from normalized invocation frequencies, as shown in Equation (6):

$$P_{ij} = (\mathbf{P})_{ij} = \frac{\text{count}(C_i \rightarrow C_j)}{\sum_{k=1}^n \text{count}(C_i \rightarrow C_k)}, \quad (6)$$

where n is the total number of classes. Classes that never call another class are modeled as absorbing states with $P_{ii} = 1$. The resulting transition matrix $\mathbf{P}$ is row-stochastic. For an absorbing DTMC, $\mathbf{P}$ can be arranged in canonical form as shown in Equation (7).

$$\mathbf{P} = \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{R} & \mathbf{Q} \end{bmatrix}, \quad (7)$$

The fundamental matrix $\mathbf{F} = (\mathbf{I} - \mathbf{Q})^{-1}$ yields the expected number of visits to each transient state before absorption. Formally, the value of F_{ij} is the expected number of times that the program enters class C_j when it is initially started from C_i . These visit expectations identify areas of good or poor execution.

The raw visit count is multiplied with the per-method timings t_k to create an impact score for each class, as per Equation (8):

$$\text{Impact}(C_i) = \sum_j F_{ij} \cdot \mathbb{E}[\text{DET}_{C_j}] + \gamma \cdot \text{Centrality}(C_i). \quad (8)$$

The first term sums up the likely contribution of DET along paths that flow through the class C_i ; the second term introduces a structural centrality measure (e.g., betweenness or PageRank) to account for the importance of the structure but not included in the static invocation counts. If a single entry class is used, the scores of the corresponding entry row of $\mathbf{F}$ can be used to get entry-state impact scores.

The CDG/DTMC stage also finds strongly connected components (SCCs). An SCC is a maximal set of classes in which it is possible to reach the other classes by directed call paths. These cycles are tightly coupled execution regions. As discussed by Parsa and Bushehrian [2], splitting such cycles across modules can create circular-communication or deadlock risks. DTMC-GA thus interprets the SCCs as atomic grouping constraints. The flow of Phase 2 has been rendered in Figure 4.

Example 2 (DTMC construction). Suppose the following set of classes is given, A, B, C, D, E, F, G, H and the call frequencies are taken from the pseudocode in Figure 1. We then proceed according to the methodology and first calculate transition probabilities P_{ij} from Equation (6), and then transform the absorbing DTMC to the canonical form of Equation (7), where we can then calculate the fundamental matrix $\mathbf{F}$ and get expected visit counts. These counts are directly used for calculating the impact score (Equation (8)) with Class A being the major hot spot because of the high local time and visit count. This example passes the SCC detection and does not have any atomic grouping constraints.

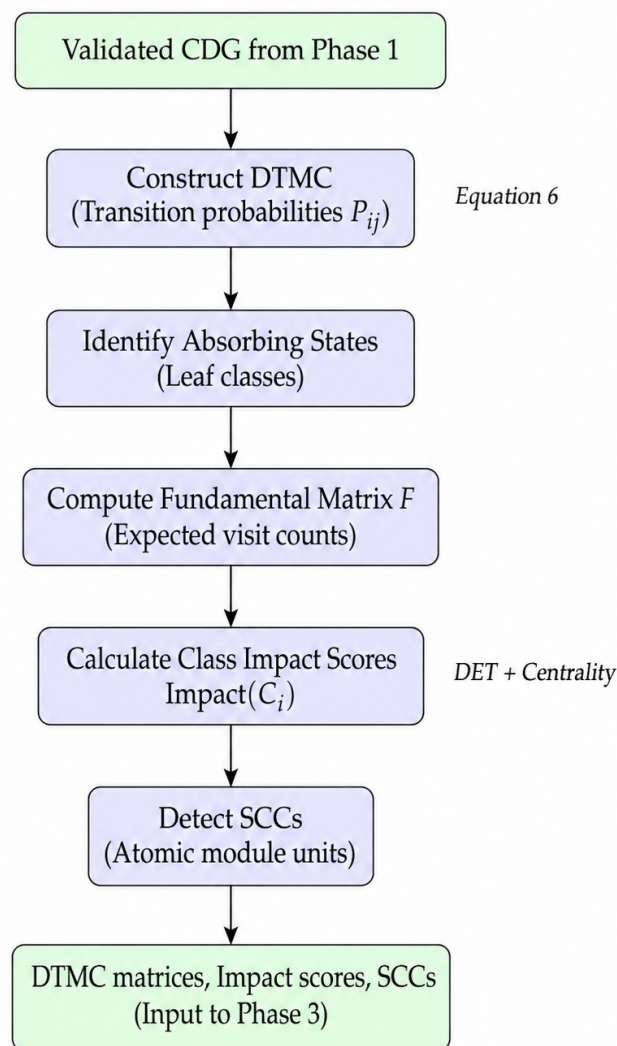


Figure 4. Phase 2 workflow for DTMC modeling and structural analysis. Purple boxes denote DTMC/structural processing steps, green boxes denote the validated input/output artifacts, and arrows show the processing sequence.

3.4. Phase 3: Distributability Enhancement Through Call Use Displacement

In many sequential programs, a method call is made just before the statement that uses the result of that method call. Call can be remote and asynchronous with no work done by the caller during the overlap window. The call-use displacement stage treats this case with caution: it looks for independent work which can be inserted between the call and the first use, increasing the t_{gap} term in the DET model in Equation (2).

A conservative, fail-closed decision policy is used to implement the call-use displacement component. Each instruction is classified as a call, a use, or an ordinary computation. A candidate is only considered eligible if the recorded safety conditions show that moving the call relative to its first use does not violate data dependencies, anti-dependencies, output or control dependencies, pointer and/or reference aliasing requirements, side-effect ordering, exception ordering, object-lifetime and/or destructor behavior, volatile/atomic ordering, or synchronization requirements before first use. If any of the required conditions are false, or if it is not possible to establish any of these conditions from the available safety metadata, the candidate is rejected. Modeled overlap is therefore only assigned to candidates for which all the required guards are satisfied. Figure 5 summarizes the workflow.

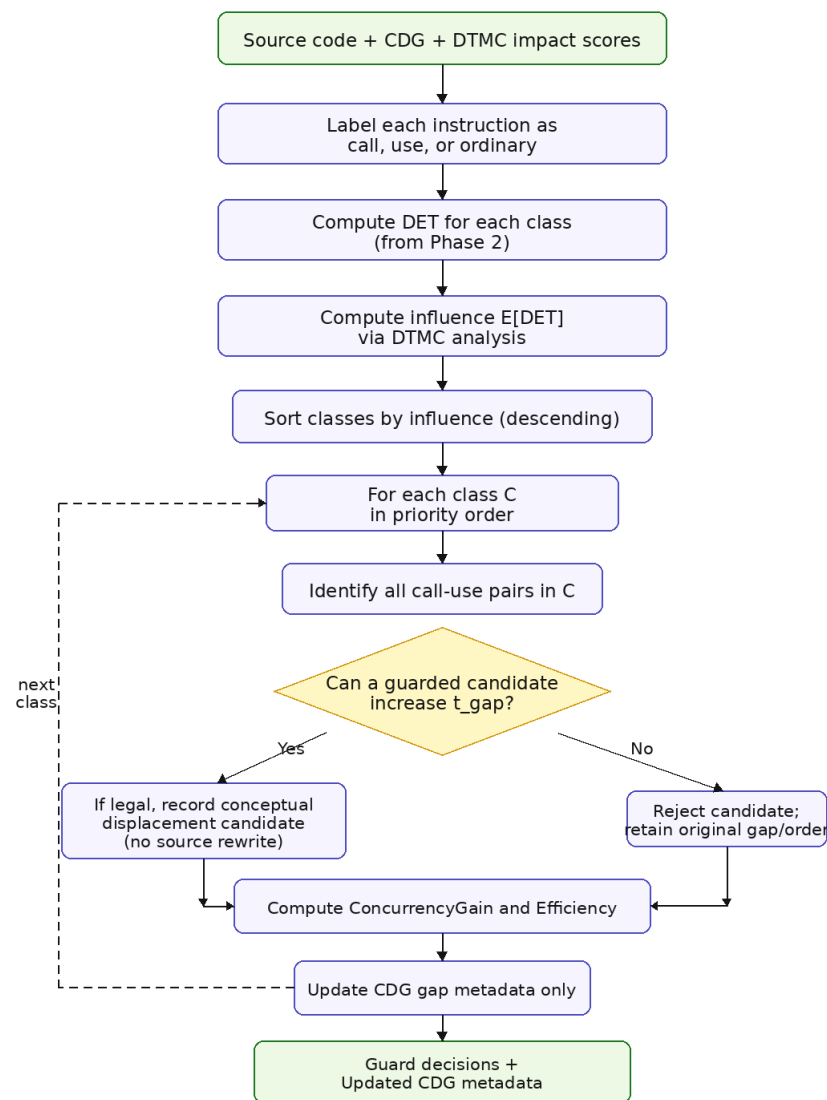


Figure 5. Workflow of the guarded call-use displacement stage. Purple boxes denote processing steps, the yellow diamond denotes the eligibility decision, green boxes denote accepted input/output states, solid arrows show control flow, and the dashed return arrow denotes iteration to the next class.

Algorithm 2 formalizes the transformation. The input consists of the source code, the current Class Dependence Graph (CDG), and the entry-state expected visit counts m_i (and associated variances) obtained from the absorbing DTMC of Phase 2. The influence of a class C_i is defined as shown in Equation (9):

$$\text{Inf}(C_i) = m_i \cdot \mathbb{E}[\text{DET}_{C_i}] + \lambda \cdot \sigma_i^2, \quad (9)$$

where $\mathbb{E}[\text{DET}_{C_i}]$ is the current estimate of its distributive execution time and σ_i^2 is the variance of the number of visits. Classes with higher influence contribute more to the overall execution time and are therefore processed first, increasing the expected impact of the displacement.

For each call-use pair (c, u) inside a method, the algorithm constructs the data- and control-dependence subgraph of the instructions between c and u . Let I_{indep} be the set of instructions that are (i) not reachable from the call result via data dependence, and (ii) not involved in any anti-dependence that would be violated by reordering. The candidate gap after displacement is calculated using Equation (10):

$$t'_{\text{gap}} = t_{\text{gap}}^{\text{orig}} + \sum_{i \in I_{\text{indep}}} \text{duration}(i). \quad (10)$$

When t'_{gap} exceeds $t_{\text{gap}}^{\text{orig}}$ and all safety guards pass, the candidate is treated as an eligible displacement opportunity. In the retained executable artifact, this decision is represented conservatively by updating the modeled call-use gap rather than by automatically rewriting arbitrary C++ source. The updated t_{gap} value is then attached to the relevant CDG edge. This process is repeated for each call-use pair in descending priority order.

Algorithm 2 Algorithm for improving class distributability

```

1: Input: Source code, CDG, visit expectations  $m_i$ , per method WCET  $\{t_k\}$ 
2: Output: Eligibility decisions and updated CDG with guarded  $t_{\text{gap}}$  values
3: for each class  $C_i$  in the system do
4:   Compute influence  $\text{Inf}(C_i)$ , for each class  $C_i$  in the system
5: end for
6: Sort classes by  $\text{Inf}(C_i)$  descending  $\rightarrow$  priority list  $L$ 
7: for each  $C$  in  $L$  do
8:   Label each instruction in  $C$  as call, use, or ordinary
9:   for each call-use pair  $(c, u)$  in  $C$  do
10:    Build data/control dependence graph for the region between  $c$  and  $u$ 
11:    Identify independent instructions  $I_{\text{indep}}$  that can be moved
12:    Compute  $t'_{\text{gap}}$  using Equation (10)
13:    if  $t'_{\text{gap}} > t_{\text{gap}}^{\text{orig}}$  then
14:      Mark the call-use pair as eligible under the recorded safety guards
15:      Update the modeled CDG gap:  $t_{\text{gap}}(C \rightarrow \text{callee}) \leftarrow t'_{\text{gap}}$ 
16:      Compute metrics: ConcurrencyGain, Efficiency
17:    else
18:      No displacement (keep in original order)
19:    end if
20:   end for
21: end for
22: return guarded eligibility decisions and the updated CDG
  
```

The guarded displacement mechanism should not be considered as a general-purpose automatic C++ source-to-source rewriter. When the recorded safety conditions show that a candidate is eligible, the retained implementation updates the modeled call-use gap accordingly. It does not try to infer arbitrary semantics of C++ programs beyond the metadata it represents and does not guarantee complete pointer analysis or semantic preservation for arbitrary production-code transformations. There is a clear separation between modeled displacement eligibility and correctness of general source rewriting, which is maintained throughout the evaluation.

Example 3 (Call-use displacement application). We illustrate the transformation on class B of the running example. The original loop body contains two call-use pairs. The DTMC analysis yields $\mathbb{E}[\text{DET}_G] > \mathbb{E}[\text{DET}_H]$, so the algorithm attempts to displace the call to G first. However, the dependency graph reveals that no independent instruction exists between the call and its use. For the call to H , the assignment $d = v(t10)$ is data-independent of the call result. Swapping the call with this assignment yields the code shown in Figure 6b. The gap now contains the computation $d = v$, giving $t'_{\text{gap}} = c_{B,10}$. After this displacement, the DET contribution of H to B is reduced because a larger portion of the waiting interval can overlap with independent work. As a result, the evaluator reports a higher modeled speedup for the revised arrangement.

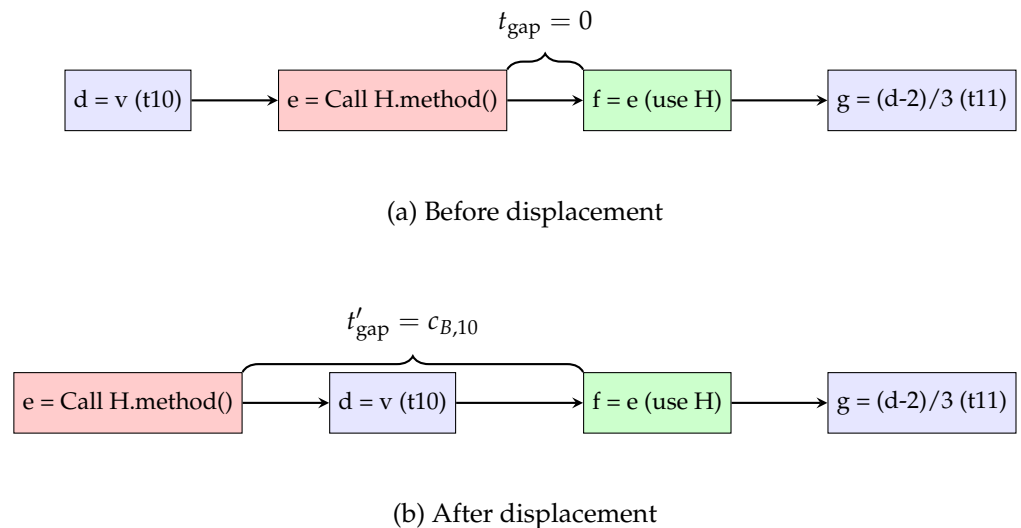


Figure 6. Call-use displacement for edge $B \rightarrow H$. Moving an independent assignment between the call and use increases the overlap window $c_{B,10}$. Blue boxes denote ordinary computations, red boxes denote the remote call, green boxes denote the first use of the returned value, and arrows indicate execution order.

Effectiveness Metrics

Two simple metrics capture the global benefit of the displacement phase. Let $\mathcal{U}$ be the set of all asynchronous call-use pairs in the system, and let $\mathcal{U}_{\text{imp}} \subseteq \mathcal{U}$ be those for which a displacement was actually performed.

$$\text{ConcurrencyGain} = \frac{|\mathcal{U}_{\text{imp}}|}{|\mathcal{U}|}, \quad (11)$$

i.e., the fraction of pairs that received a non-zero gap. For a pair $u \in \mathcal{U}$, if $t_{\text{gap}}^{\max}(u)$ is the maximum possible gap (the sum of all independent instructions between the original call and use), the Displacement Efficiency of that pair is $t'_{\text{gap}}(u) / t_{\text{gap}}^{\max}(u)$. The overall efficiency is the average over $\mathcal{U}_{\text{imp}}$.

In Example 3, $\mathcal{U}_{\text{imp}}$ contains the pair $B \rightarrow H$, giving $\text{ConcurrencyGain} = 1/2$ (only H was improved) and an efficiency of 1.0, because all available independent instructions were moved into the gap.

After this phase, eligible CDG edges contain enlarged t_{gap} values, which feed larger overlap windows into the DET model. The updated representation is then passed to the final GA-based modularization stage (Phase 4).

3.5. Phase 4: Genetic Algorithm for Concurrency-Aware Modularization

Selecting module boundaries that expose concurrency while respecting deployment constraints is a combinatorial search problem and is NP-hard in the general case [43]. DTMC-GA therefore uses a tailored genetic algorithm whose encoding, constraint handling, and fitness function are designed for performance-aware architecture extraction.

3.5.1. Encoding and Search Space Reduction

Each candidate solution is encoded as an integer string $S = [s_1, s_2, \dots, s_n]$, where $s_i \in \{1, \dots, m\}$ assigns class i to module s_i , and the maximum number of modules is capped. In order to reduce the search space, the executable module cap reduces both the labeled chromosome space and the number of distinct unlabeled partitions. The module cap for the 12-class `layered_services_12` fixture is $m = 3$. There are $12^{12} = 8,916,100,448,256$ possible assignments in the unrestricted labeled assignment space but only $3^{12} = 531,441$

possible assignments with three module labels. If module-label symmetry is removed, the unrestricted number of distinct partitions is the Bell number $B_{12} = 4,213,597$, while the cap admits 88,574 distinct partitions. This is a reduction of approximately 97.90% in the unrestricted partition space. The reduction is meant to show the combinatorial effect of the module cap and should not be taken as proof of GA scalability.

3.5.2. Fitness Evaluation

The fitness of a modularization S is evaluated using a weighted combination of performance and structural metrics:

$$f(S) = w_1 \cdot \text{MQ}_{\text{speedup}}(S) - w_2 \cdot \text{CommCost}(S) + w_3 \cdot \text{LoadBalance}(S) - w_4 \cdot \sum_{r=1}^c \text{Penalty}_r(S). \quad (12)$$

The components are:

- $\text{MQ}_{\text{speedup}}$ —speedup-aware Modularization Quality. For a single asynchronous method call we define a Method-Level Speedup S_m as the difference between serial and parallel execution times, as formulated in Equation (13):

$$S_m = T_{\text{serial}} - T_{\text{parallel}} \quad (13)$$

Summing S_m over all calls from class C_i to C_j yields the Inter-Class Speedup $S(C_i, C_j)$. When two classes reside in different modules, this speedup is realizable. The Inter-Module Speedup between modules M_i and M_j averages the inter-class speedups over all cross-module class pairs, as shown in Equation (14):

$$S(M_i, M_j) = \frac{1}{|M_i| |M_j|} \sum_{\substack{C_p \in M_i \\ C_q \in M_j}} S(C_p, C_q). \quad (14)$$

Finally, the speedup-aware MQ sums the inter-module speedups over all distinct module pairs, as defined in Equation (15):

$$\text{MQ}_{\text{speedup}}(S) = \sum_{i=1}^{k-1} \sum_{j=i+1}^k S(M_i, M_j). \quad (15)$$

- CommCost —Communication Overhead, defined as the fraction of the system's total communication cost that crosses module boundaries.
- LoadBalance —Load Balance Quality, computed as $1 - C_v$, where C_v is the coefficient of variation of the estimated module execution times.
- Penalty_r —piecewise quadratic penalty terms that enforce deployment constraints (module size, co-location requirements, resource budgets).

3.5.3. Genetic Algorithm Pseudocode

The GA follows the standard evolutionary loop (see Figure 7) and is detailed in Algorithm 3. It uses tournament selection, uniform class-wise crossover, random one-class mutation, and two-elite retention; each candidate is evaluated using the fitness function in Equation (12).

Algorithm 3 Genetic algorithm for constrained concurrency-aware modularization

Require: CDG $G = (V, E)$, maximum modules m , population size N_{pop} , maximum generations G_{max}

Ensure: Best-found modularization S^*

- 1: Create initial population $\mathcal{P}$ of N_{pop} random valid encodings.
- 2: Evaluate fitness $f(S)$ for each $S \in \mathcal{P}$ using Equation (12).
- 3: $S^* \leftarrow \arg \max_{S \in \mathcal{P}} f(S)$; $gen \leftarrow 0$.
- 4: **while** $gen < G_{\text{max}}$ **do**
- 5: Sort $\mathcal{P}$ by fitness and copy the two best individuals to $\mathcal{P}'$.
- 6: **while** $|\mathcal{P}'| < N_{\text{pop}}$ **do**
- 7: Select parents p_1 and p_2 by tournaments of size 4.
- 8: **Uniform crossover:** for each class, choose its module from p_1 or p_2 with probability 0.5.
- 9: **Mutation:** with probability 0.30, randomly select one class and reassign it to a valid module.
- 10: Add the offspring to $\mathcal{P}'$.
- 11: **end while**
- 12: Evaluate $f(S)$ for each $S \in \mathcal{P}'$ and set $\mathcal{P} \leftarrow \mathcal{P}'$.
- 13: $S_{\text{best}} \leftarrow \arg \max_{S \in \mathcal{P}} f(S)$.
- 14: **if** $f(S_{\text{best}}) > f(S^*)$ **then**
- 15: $S^* \leftarrow S_{\text{best}}$.
- 16: **end if**
- 17: $gen \leftarrow gen + 1$.
- 18: **end while**
- 19: **return** S^*

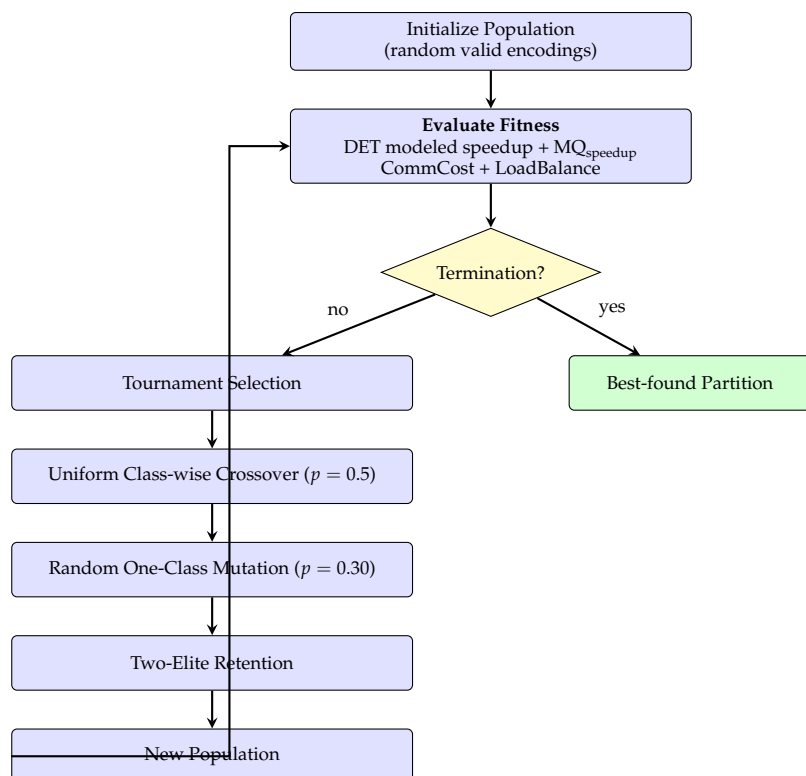


Figure 7. Phase 4 evolutionary cycle for GA-based modularization. Blue boxes denote GA processing steps, the yellow diamond denotes the termination decision, the green box denotes the returned best-found partition, and arrows show control flow; the return arrow from the new population indicates iteration to the next generation.

3.5.4. Genetic Operators and Evolution

Initialization. The initial population is created by randomly assigning each class to a module in $\{1, \dots, m\}$, producing random valid partition encodings.

Selection. Tournament selection samples four individuals uniformly at random, and the best individual in each tournament is selected as a parent.

Crossover. Uniform class-wise crossover selects the module assignment for each class independently from either parent with probability 0.5.

Mutation. With probability 0.30 for each offspring, one class is selected at random and reassigned to a randomly selected valid module.

Elitism and diversity. The two best individuals are retained unchanged in each generation. Population diversity is recorded from pairwise differences in module assignments.

3.5.5. Termination and Scalability

The algorithm stops when the fixed maximum generation budget $G_{\max}$ is reached. For systems beyond the released fixtures, incremental fitness evaluation and parallel population evaluation remain possible scalability strategies, but their effect on larger projects requires separate empirical validation.

3.6. Implementation and Practical Considerations

Figure 8 depicts the architecture that was intended. The design includes a combination of a parser (CLANG/LLVM), CDG (Construction of Dependency Graphs), DTMC (Discrete-Time Markov Chain) analysis, guarded code enhancement and GA optimization. The quality indicators are collected by a configuration manager, which provides user-defined parameters and constraints. This artifact has been released and consists of the reproducible downstream benchmark-suite pipeline: CSV fixture inputs, raw results, statistical scripts, generated figures, Docker configuration and CI verification.

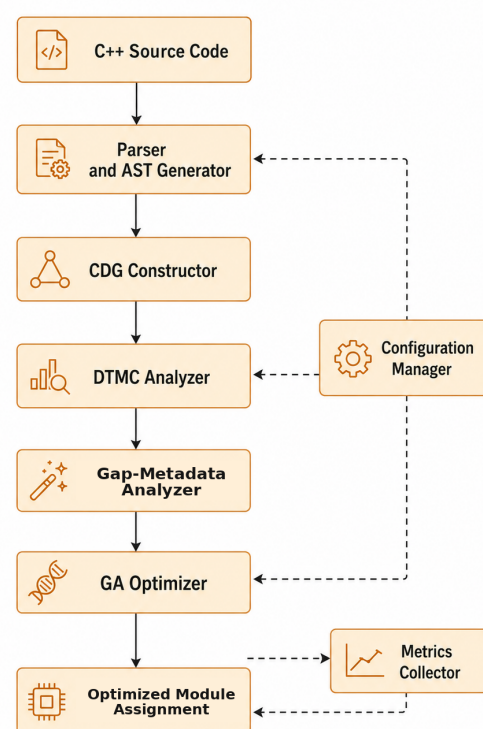


Figure 8. Implementation architecture of the DTMC-GA methodology. Colored boxes distinguish the main architectural components; solid arrows indicate the primary processing/data path, while dashed arrows indicate configuration, metric, or control feedback.

The source-analysis stage, at the methodological level is specified by Algorithm 1: units to be analyzed are given semantic information, units are instantiated, virtual-method resolution occurs if it is possible. The resulting CDG is built with weighted edges, validated for consistency, and enriched with metadata. For the optimization phase, the retained artifact uses a population size of 12 over 8 generations, tournament size 4, uniform class-wise crossover with a 0.5 parent-choice probability, random one-class mutation with event probability 0.30, and two-elite retention.

For larger systems, several performance strategies are relevant. Incremental analysis can reduce repeated CDG construction, GA fitness evaluation can be parallelized across candidate partitions and seeds, and memory usage is strongly influenced by the DTMC matrix representation. The revision therefore adds a controlled sparse-graph scalability stress test up to 1000 classes using sparse CSR propagation. This test addresses computational feasibility under a fixed search budget; it does not establish production-scale search adequacy or optimality.

4. Experimental Design and Reproducibility Protocol

The evaluation design is stated before the numerical results so that the scope of the claims is clear. The fundamental unit of analysis is a fixture–algorithm–seed run. This arrangement helps to ensure standardization of inputs across methods, maintains the stochastic variation and prevents cherry picking and best run selection.

4.1. Benchmark and Workload Reporting

The released artifact evaluates DTMC-GA on five local executable benchmark fixtures. Each fixture includes class metadata, weighted call edges, conservative call-use safety decisions, and a documented subsystem model. These fixtures exercise the end-to-end pipeline, but they are not a substitute for traced industrial systems. The manuscript therefore reports them explicitly as local fixture-suite evidence. The fixture inventory is given in Table 3.

Table 3. Benchmark-suite fixtures in the released reproducibility artifact. Edge and safety counts are computed from the included CSV fixture files and are reported to make the local evaluation boundary explicit.

Fixture	Classes	Edges	Weighted Calls	Async Edges	Safe Pairs	Subsystem Model
demo_eight_class	8	12	72	5	5	running example
layered_services_12	12	28	162	7	7	layered service
event_pipeline_16	16	35	190	8	7	event processing
cyclic_workflow_20	20	43	222	11	10	workflow core
plugin_platform_24	24	60	289	17	15	plugin runtime

The five fixtures released are not an industrially representative sample of production C++ systems, but instead provide a controlled set of systems. They form five architectural patterns: a running example, layered services, an event-processing pipeline, a cyclic workflow, and a plugin platform. Across these cases, the number of classes, call edges, weighted call activity, opportunities for asynchronous calls, topology, communication characteristics, and accepted or rejected safety cases vary. The 30-seed protocol is the same for all of the compared methods, with controlled variation of the different structural, communication, overlap, and safety conditions.

The fixture metadata therefore show that the evaluation deliberately exercises increasingly larger local dependency structures while remaining within a bounded, reproducible

benchmark-suite. The OpenCV checkpoint described below adds a bounded real-project calibration layer, while broader multi-project validation remains future work.

4.2. Claim–Validation Boundary

The claim–validation boundary in Table 4 is included to keep interpretation of the results consistent with the level of evidence available in the study. The released artifact supports reproducibility and model-consistency claims for the fixture suite, whereas the OpenCV 4.7.0 checkpoint adds a bounded real-runtime calibration. This distinction is part of the experimental design because the measured OpenCV comparator is OpenCV’s native threaded implementation rather than a source rewrite generated by DTMC-GA. Therefore, the study does not make a direct claim about production-runtime speedup of DTMC-GA on OpenCV.

Table 4. Claim–validation boundary used to interpret the evidence in this manuscript.

Validation Level	Evidence Available in This Version	Claim Permitted
Formal model	Formal DTMC, DET, penalty, and GA definitions	The optimization objective is well-defined and combines execution likelihood, overlap, communication cost, load balance, and constraints.
Executable fixtures	Five local fixtures, 750 retained runs, raw CSV outputs, scripts, figures, Docker, and CI	DTMC-GA is reproducible on the released fixtures and shows fixture-bounded tendencies under the implemented cost model.
Surrogate consistency	Predicted-versus-generated artifact outputs and statistical summaries	DET- and MQ-based indicators are internally consistent for the artifact pipeline, but remain surrogate indicators.
External runtime calibration	OpenCV 4.7.0 matched runtime checkpoint and executed instrumentation	A measured native-OpenCV calibration point is reported; it is not measured speedup of a DTMC-GA-generated source rewrite.
Industrial deployment	Not included in this version	No claim is made about production adoption, developer acceptance, or unsupervised deployment safety.

This boundary directly addresses the main generalization risk. The present paper can claim that the method, implementation pipeline, retained-run analysis, controlled scalability campaign, and bounded OpenCV calibration are reproducible within their stated scope. It cannot claim direct measured runtime improvement from a DTMC-GA-generated production rewrite until a matched optimized executable is produced and evaluated under project-specific workloads, hardware, compiler settings, and runtime measurements.

4.3. Compared Baselines

The baseline set used in the released artifact consists of Bunch, FCA, a random valid partition, and traditional MQ optimization. The Bunch-style comparison wrapper maintains a fixed set of parameters for all retained fixtures: a population size of 12, 8 generations, a tournament size of 4, 2 elites, uniform per-gene crossover, and a mutation event probability of 0.30, with the same executable module cap for all comparisons. There is no fixture-specific tuning. The FCA-style wrapper is deterministic, with the classes ordered by weighted structural degree and then assigned using a round-robin process under the same module cap. The FCA-style procedure is thus free of stochastic hyperparameters that need to be calibrated per fixture.

4.4. Statistical Protocol

Each stochastic algorithm in the released benchmark-suite evaluation is executed for 30 independent runs with distinct random seeds; with five fixtures, each algorithm contributes 150 retained rows to the aggregated summary. Summary tables report aggregated descriptive statistics, and best-run filtering is not used. Statistical significance is assessed with the Friedman test across methods, followed by Bonferroni-corrected Wilcoxon signed-rank tests for pairwise comparisons. Effect sizes are reported using rank-biserial correlation. In the Section 5, the descriptive table is therefore paired with a compact statistical-support table so that the reader can distinguish mean differences from statistically supported fixture-suite tendencies. The raw per-run results and statistical scripts are included in the public artifact.

4.5. Reproducibility Artifact Requirements

The public artifact contains the implementation, benchmark metadata, workload descriptions, configuration files, raw results, processed results, statistical scripts, figure-generation scripts, Dockerfile, license, citation file, README, and the GitHub Actions workflow used for reproducibility checks. It is available at <https://github.com/dhok8400-png/DTMC-GA-Reproducibility-Artifact> (accessed on 31 August 2026) and archived on Zenodo with DOI: 10.5281/zenodo.21045304.

4.6. Same-Objective GA+DET Control

An additional same-objective control is used to examine how much DTMC visit weighting contributes. This control employs the same GA engine, DET evaluator, modeled-speedup fitness, communication and load-balance terms, guarded displacement decisions, module cap, and the same 30 random seeds as DTMC-GA. The difference between the two comes only from the execution-frequency weighting provided to the evaluator. DTMC-GA employs finite-horizon DTMC expected visits, whereas the control assigns a uniform one-visit prior to each class. Maintaining the same search process and objective allows this design to test only the effect of the DTMC visitation model.

4.7. NSGA-III Comparator

A many-objective NSGA-III comparator is also included to reduce reliance on comparisons with older structural modularization techniques. The same fixtures, DTMC-DET inputs, constraints, random seeds, population size, and generation budget are used for the comparator as for the retained DTMC-GA configuration. NSGA-III does not combine all criteria into a single scalar objective during environmental selection; rather, it treats modeled speedup, communication overhead, load balance, and the modeled-speedup-aware modularization measure as separate objectives. The aim of this experiment is to provide a more up-to-date search-based reference for evaluating the retained GA scalarization.

4.8. Coefficient Sensitivity

In the sensitivity analysis, one active objective coefficient is changed at a time while the rest of the experimental settings are kept constant. Each coefficient is evaluated at $0.50\times$, $0.75\times$, $1.00\times$, $1.25\times$, and $1.50\times$ its retained value. The DTMC visitation inputs and all other experimental settings, including the GA configuration, module cap, safety decisions, fixtures, and random seeds, remain unchanged across these runs. The aim of this experiment is to investigate the robustness of the main conclusions around the retained coefficient configuration. The analysis is neither intended to demonstrate that the selected coefficient values are globally optimal nor that they can be applied to other workloads.

4.9. Controlled C++ Equivalence Harness

A controlled C++ equivalence harness is used to test source-level safety separately from the fixture metadata. An allowed call motion is compared with the original ordering over 200,000 deterministic inputs using GCC and Clang, including optimized and AddressSanitizer/UndefinedBehaviorSanitizer-enabled builds. There are also intentionally unsafe reorderings included in the harness, such as pointer or reference aliasing, side-effect ordering, object-lifetime or destructor ordering, and exception ordering. The negative cases are added to make sure that the guarded hazards can influence the observable behavior of the program and therefore have to be rejected by a conservative transformation policy.

4.10. Scalability Protocol

Deterministic sparse stress graphs with 50, 100, 250, 500, 750, and 1000 classes are used to examine computational scalability. Thirty random seeds are executed for each graph size. The DTMC-related calculations use sparse CSR propagation to avoid artificially measuring the quadratic storage requirements of a dense transition matrix. End-to-end analysis time and peak resident memory are recorded for every size. This experiment is designed to assess computational feasibility under a fixed search budget and sparse topology; it is not intended to show that the fixed GA budget is sufficient or optimal for a production-scale system.

4.11. OpenCV Runtime-Calibration Protocol

The OpenCV 4.7.0 checkpoint uses the same source commit reported in the Introduction together with a fixed five-convolution DNN workload. Runtime calibration uses 30 matched pairs to compare single-threaded execution with OpenCV's native parallel execution. Instrumentation from the same workload is reduced to a bounded empirical DTMC-GA/DET representation and evaluated over 30 seeds. The purpose of this protocol is to compare the magnitude of the model-derived speedup with a measured native-runtime reference. The measured parallel executable is OpenCV's native threaded implementation rather than a source rewrite generated by DTMC-GA.

5. Results

The results report the executable benchmark-suite evidence included in the public artifact together with the revision-time same-objective control, NSGA-III comparator, safety-validation harness, scalability campaign, and bounded OpenCV runtime calibration. The evidence boundary remains explicit: fixture-suite speedups are modeled quantities, while the OpenCV $1.8691 \times$ value is a measured native-runtime calibration point and not measured DTMC-GA source-rewrite performance.

5.1. Executable Benchmark-Suite Results

The benchmark-suite layer uses local CSV fixtures to exercise the executable pipeline: normalized class tables, weighted call edges, conservative call-use safety decisions, DTMC visit estimation, DET-inspired fitness evaluation, GA and baseline optimization, retained per-run outputs, statistical scripts, and figure generation. The resulting rows are generated by the released artifact pipeline rather than by third-party large-system runtime measurements. They should therefore be read as reproducibility and pipeline-validation evidence, not as industrial evidence for Chromium, Firefox, MySQL, or other external projects.

The retained runs use the five-item benchmark inventory reported in Table 3.

The benchmark-suite run generated 750 retained experiment rows: five benchmark fixtures, five algorithms, and 30 seeds per algorithm. Best-run filtering was not used. Table 5 summarizes the retained raw outputs aggregated over all included benchmark fixtures.

Table 5. Benchmark-suite summary over all fixtures and retained seeds (mean $\pm$ SD), regenerated from `raw_results/experiment_runs.csv`.

Algorithm	Modeled Speedup	Communication Overhead	Load Balance	MQ
DTMC-GA	2.708 ± 0.329	0.165 ± 0.053	0.849 ± 0.111	18.741 ± 6.151
Bunch	2.143 ± 0.564	0.088 ± 0.054	0.560 ± 0.267	8.180 ± 6.599
FCA	2.146 ± 0.471	0.181 ± 0.047	0.683 ± 0.162	14.858 ± 8.686
RandomValidPartition	2.093 ± 0.420	0.168 ± 0.056	0.548 ± 0.213	15.113 ± 7.015
TraditionalMQ	2.217 ± 0.303	0.159 ± 0.049	0.597 ± 0.186	15.766 ± 4.675

Table 6 adds the corresponding statistical support for the speedup comparison. For each fixture, it reports the global Friedman test across all five methods and the Bonferroni-adjusted Wilcoxon comparison between DTMC-GA and the strongest non-DTMC-GA modeled-speedup baseline in that fixture. The table is intentionally compact: it is not presented as external runtime evidence, but it makes the retained-run analysis more transparent than a mean-only comparison.

Table 6. Fixture-level statistical support for the benchmark-suite modeled-speedup comparison. The pairwise test compares DTMC-GA with the strongest non-DTMC-GA modeled-speedup baseline in the same fixture. Values are regenerated from `processed_results/statistical_tests.csv`.

Fixture	Friedman p	Strongest Comparator	Adjusted p	$ r_{rb} $
cyclic_workflow_20	2.53×10^{-15}	Bunch	4.97×10^{-4}	0.789
demo_eight_class	8.40×10^{-21}	TraditionalMQ	2.38×10^{-4}	0.875
event_pipeline_16	5.77×10^{-13}	Bunch	1.23×10^{-3}	0.755
layered_services_12	1.48×10^{-12}	Bunch	2.02×10^{-2}	0.626
plugin_platform_24	2.50×10^{-17}	FCA	3.86×10^{-6}	0.927

In the revised manuscript, the evidence-scope and statistical-summary tables are given separate roles. Table 4 shows the single claim-validation boundary between quantities derived from the model, controlled validation evidence, and measured runtime evidence. Table 6 is used for statistical support at the fixture level, including Friedman tests and Bonferroni-adjusted paired Wilcoxon comparisons. This separation helps prevent duplication between methodological evidence boundaries and the reporting of statistical results.

Within the released fixture-suite, DTMC-GA reports the highest mean modeled speedup and load-balance values among the five released comparison methods, while Bunch reports the lowest mean communication overhead. The statistical-support table indicates that the speedup differences are not produced by selecting isolated favorable runs; retained-run comparisons remain significant after Bonferroni correction against the strongest non-DTMC-GA speedup comparator in each fixture. Interpretation does not go beyond what is deliberately contained. DTMC-GA is not the best with all measures. Rather, it will allow for further communication, if the DTMC-DET scoring model indicates that the overlap and load balance can offset that cost. This is a fixture based claim—it's not a universal claim across workloads or deployments.

Table 7 summarizes the validation diagnostics used to interpret Figures 9 and 10. These diagnostics are useful for checking internal consistency of the artifact pipeline, but the measured values remain surrogate outputs from the released fixture-suite rather than third-party runtime measurements.

Table 7. Internal validation diagnostics for the released benchmark-suite artifact. The reported values summarize artifact-generated surrogate outputs and should not be read as measured external runtime validation.

Diagnostic	<i>n</i>	Pearson <i>r</i>	Spearman ρ	Error/ R^2	Scope
Predicted vs. surrogate modeled speedup	750	0.996	0.995	RMSE = 0.041; MAPE = 1.45%	Artifact surrogate
Speedup-aware MQ vs. surrogate modeled speedup	750	0.709	0.694	$R^2 = 0.502$	Artifact surrogate
Traditional MQ vs. surrogate modeled speedup	750	−0.424	−0.351	$R^2 = 0.180$	Artifact surrogate

Figures 9–16 visualize benchmark-suite outputs regenerated from retained CSV files, including fixture-level behavior with dispersion, validation consistency, ablation effects, latency sensitivity, and diagnostic partition structure.

The diagnostic figures have been created so that their interpretation does not depend solely on color. Different marker and line styles, hatch patterns, clear outlines, consistent axis and legend labels, larger labels, and numerical annotations where suitable are the distinguishing features of the final plots. These design decisions make the electronic version more readable and maintain easy-to-understand comparisons in grayscale printouts.

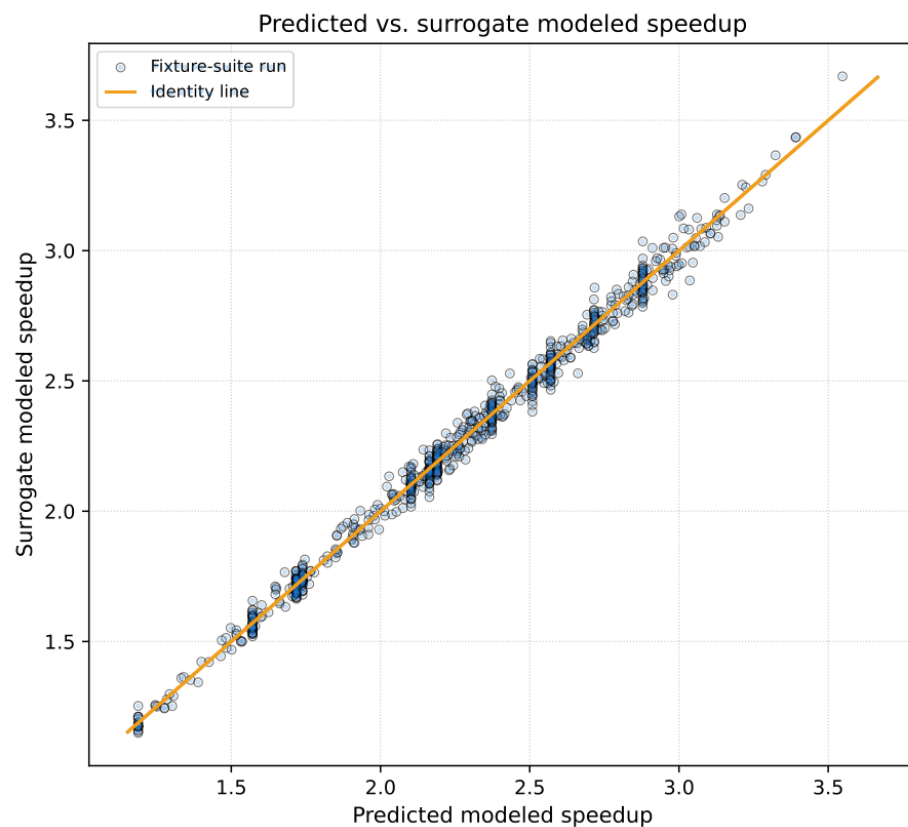


Figure 9. Predicted versus artifact-generated surrogate modeled speedup for the benchmark-suite pipeline. The scatter summarizes internal artifact consistency and is not third-party runtime evidence. Overlapping markers represent repeated or near-identical retained-run values. The solid orange line is the identity line ($y = x$).

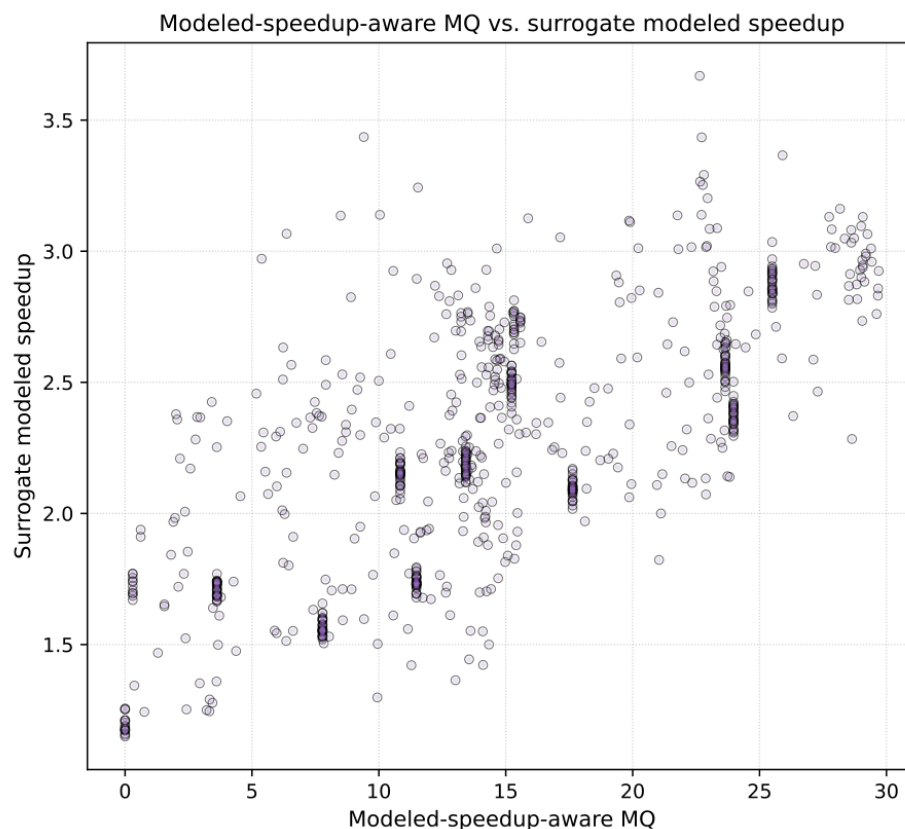


Figure 10. Speedup-aware MQ versus artifact-generated surrogate modeled speedup in the benchmark-suite run. The association is interpreted as an internal surrogate diagnostic rather than a measured runtime claim. Overlapping markers denote repeated or near-identical retained-run values and do not obscure the overall association.

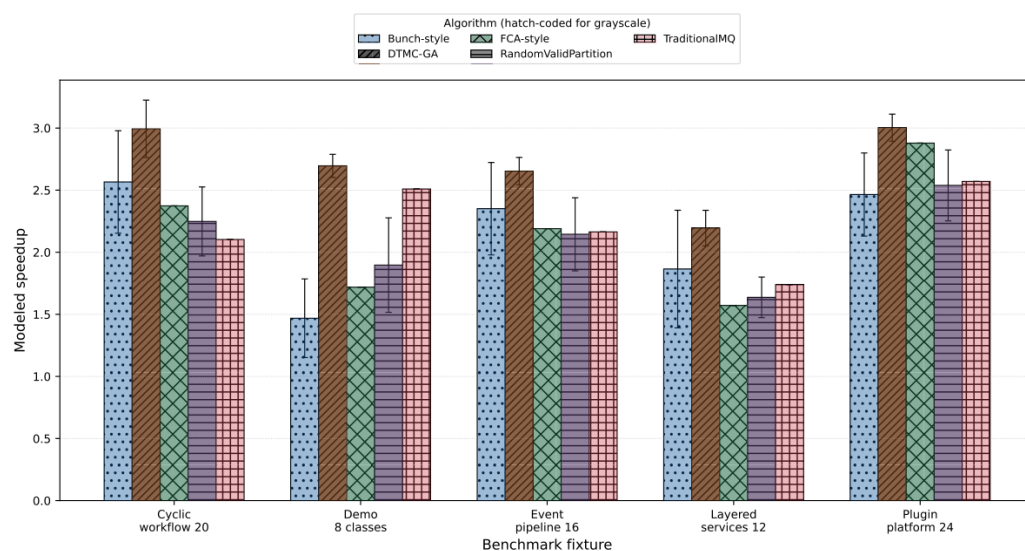


Figure 11. Fixture-level modeled speedup by algorithm. Error bars denote the retained-run standard deviation across 30 seeds per algorithm–fixture pair. Algorithm identity is encoded consistently by matching color and hatch pattern in both the bars and legend.

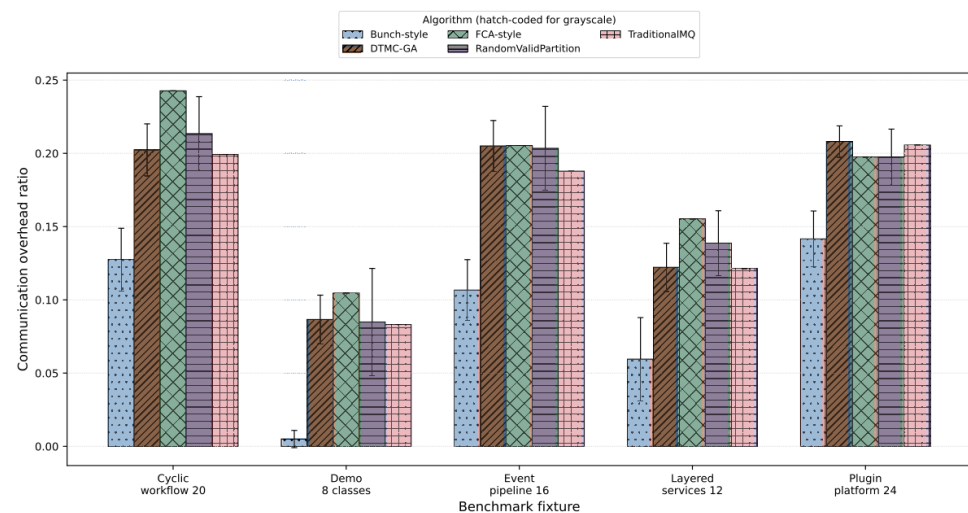


Figure 12. Fixture-level communication overhead by algorithm. Error bars denote the retained-run standard deviation across 30 seeds per algorithm–fixture pair. Algorithm identity is encoded consistently by matching color and hatch pattern in both the bars and legend.

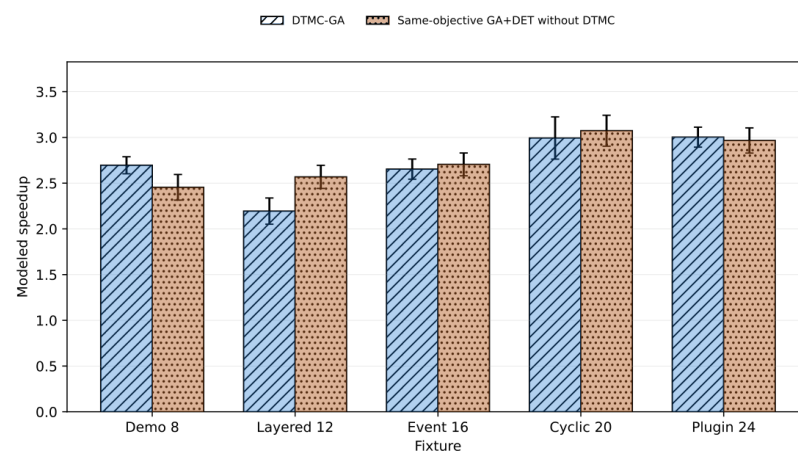


Figure 13. Ablation effects reported relative to full DTMC-GA; exact retained values are available in `raw_results/ablation_results.csv`. Method identity is encoded consistently by matching color and hatch pattern in both the bars and legend.

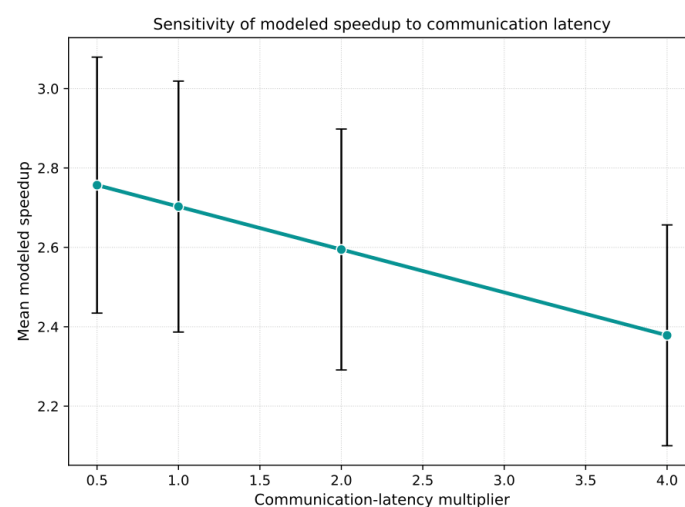


Figure 14. Sensitivity of mean modeled speedup to the communication-latency multiplier. The green line connects the mean modeled-speedup values at the evaluated multipliers, the circular markers show those means, and the vertical error bars show retained-run dispersion.

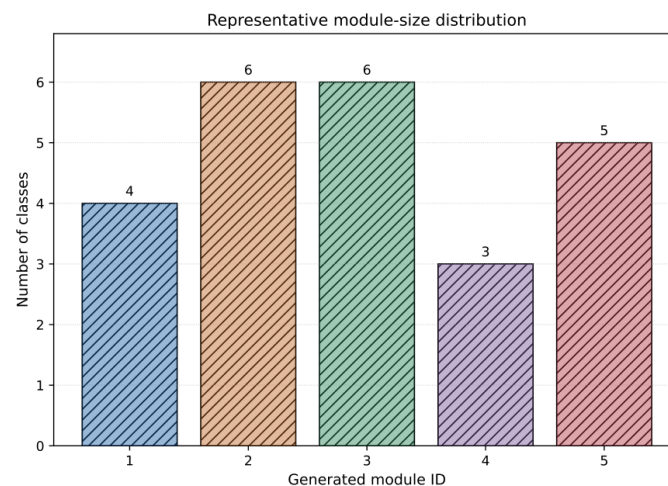


Figure 15. Representative module-size distribution for plugin_platform_24 under DTMC-GA, seed 1. This diagnostic plot illustrates one retained partition and is not used as an aggregate result.

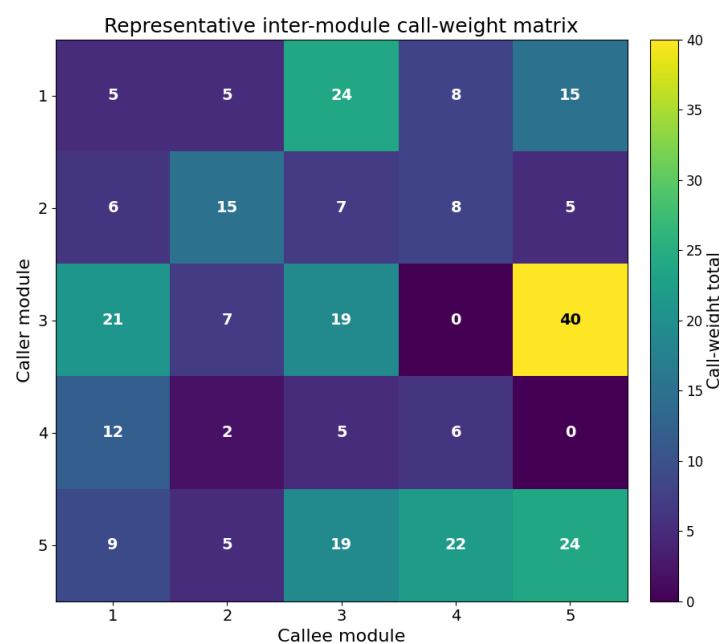


Figure 16. Representative inter-module call-weight matrix for plugin_platform_24 under DTMC-GA, seed 1. Cell values show retained call-weight totals for this diagnostic partition, and the colorbar uses the same continuous colormap as the matrix cells.

5.2. Same-Objective Control Results

For the same-objective control, no consistent benefit was found from DTMC visit weighting. DTMC-GA obtains the greater mean modeled speedup on two fixtures, while the uniform one-visit control obtains the greater mean on three fixtures. The paired Wilcoxon comparison of all 150 matched fixture-seed runs is not statistically significant, with $p = 0.0966$. The results show that the influence of DTMC visitation weighting varies with workload and is not uniformly positive under the retained modeled-speedup objective.

5.3. NSGA-III Comparator Results

The NSGA-III experiment provides a more recent comparison of the effect of the search procedure. The mean modeled speedups from the five retained fixtures are summarized as 2.737 for NSGA-III and 2.708 for DTMC-GA, showing an aggregate mean improvement of about 1.07%. However, none of the paired differences between the fixtures is statistically

significant after Bonferroni correction. The results therefore do not allow a conclusion to be drawn regarding the superiority of either optimizer under the tested conditions, but they do show that a contemporary many-objective search technique is competitive with the retained DTMC-GA setup.

5.4. Safety Validation Results

The controlled C++ equivalence experiment is designed to test output preservation for the allowed displacement represented by the harness. Across 200,000 deterministic inputs, the allowed motion produced the same output as the original ordering under GCC and Clang, including optimized and sanitizer-enabled builds. Deliberately forced reorderings involving pointer or reference aliasing, side-effect order, object lifetime or destructor order, and exception order show that these hazards can change observable program behavior. The results support the use of a fail-closed guard policy, but the evidence is limited to the controlled type of transformation represented in the harness.

5.5. OpenCV Runtime Calibration Results

For OpenCV 4.7.0, the ratio-of-means speedup of native-parallel execution relative to the matched single-threaded runs is $1.8691\times$. Instrumentation from the same workload is converted to a bounded empirical DTMC-GA/DET representation and evaluated over 30 seeds, producing a mean modeled speedup of $1.0000\times$ with an SD of 0.0000. The absolute difference between the modeled and measured speedups is 46.50%. This gap shows that the present empirical model should not be assumed to reproduce the magnitude of native OpenCV parallel performance. The measured $1.8691\times$ value is therefore used as a real-runtime calibration point and is not presented as measured performance of an optimized executable generated by DTMC-GA.

5.6. Scalability Results

Across the scalability campaign, mean end-to-end analysis time increases from approximately 0.024 s for the 50-class graph to approximately 0.286 s for the 1000-class graph. Peak process memory increases from about 119.1 MiB to 120.9 MiB. These measurements show that the sparse implementation remains computationally feasible at the reported scale under the fixed search budget and sparse topology. They do not demonstrate that partition quality remains unchanged as the search space grows. Computational tractability and optimization quality should therefore be considered separately.

5.7. Scope of External Large-Project Validation

The OpenCV experiment reduces some of the limitations associated with relying mainly on controlled fixtures. It adds real-source verification, dynamic instrumentation, runtime measurement, and model calibration using a real production software project. However, the measured parallel executable is OpenCV's native threaded implementation rather than an executable generated from a DTMC-GA partition. To support a stronger production-runtime claim, it would be necessary to evaluate a matched DTMC-GA-generated optimized implementation and test the method on additional medium- or large-scale real-world projects.

5.8. Threats to Validity

Internal validity. The main internal-validity risks are static call-frequency estimation, stochastic GA variability, implementation errors, and parameter sensitivity. These risks are mitigated by the ability of the artifact to include 30 runs per algorithm/fixture, use fixed seeds, output the raw CSV data, and regenerate tables and figures. The OpenCV

checkpoint provides one bounded measured-runtime calibration, while broader project-specific runtime studies are still required.

External validity. The executable evidence is only available in the form of local C++ fixtures acting out architectural patterns, and are not truly representative of the diversity of production systems. The dispatch, memory, and synchronization models used may be different in other languages, and for heterogeneous and cloud deployments, the terms C_{ij} and the contention terms may need to be recalibrated and workload-specific profiling may be necessary.

Construct validity. DET, speedup-aware MQ, communication overhead and load balance are proxies of the runtime-relevant behaviors. They are found to be useful depending on how well they correlate with measured execution time, prediction error and workload sensitivity. Surrogate and pipeline validation remain the main fixture-suite validation, while the OpenCV checkpoint adds a bounded runtime calibration; broader runtime correlation, contention modelling, and alternative cost models remain future validation tasks.

Conclusion validity. Fixture-suite is reproducible, but small. The number of 30 runs kept per algorithm–fixture pair removes the selection bias of best runs and the stochastic-search noise, but only results can be drawn from the data that has been released from the competition, retained and the baselines used. Further work is needed in the form of more comprehensive benchmarks, additional real projects, and broader parameter-sensitivity studies.

6. Discussion

There are some useful but limited observations to be made from the fixture-suite experiments. Performance-aware modularization is not just about reducing communication, since the value of a partition can be influenced by the likelihood of its execution, the opportunity for overlap, its load-balancing characteristics, and its structural properties. The aggregate mean modeled speedup of DTMC-GA is higher than that of any of the other released comparison methods, while Bunch has the lowest average communication overhead among the five released methods. It is important to note, however, that the results from the revision-time controls and modern comparator cannot be interpreted as evidence of universal superiority of DTMC-GA. Thus, the claim–validation boundary becomes a guide for interpreting the results rather than merely a limitation statement.

The same-objective experiment limits the strength of any general conclusion about DTMC visitation weighting. Using uniform visit weights improves the modeled result for some fixtures and worsens it for others. When all matched runs are pooled across fixtures, the overall difference is not statistically significant. The contribution of the Markov component should therefore be understood as workload dependent. Its main value is in giving the optimizer information about execution relevance rather than guaranteeing that the best partition will be created for every workload.

The NSGA-III comparison also shows that the observed performance trade-offs depend on the optimization procedure. Its aggregate mean modeled speedup is slightly higher than that of DTMC-GA, but the fixture-level differences are not statistically significant after correction for multiple comparisons. This means that good performance against older structural wrappers should not be extrapolated as evidence of general optimizer superiority. Under the retained experimental conditions, a modern many-objective search method is competitive, and no single optimization strategy is uniformly best.

The most safety-sensitive component of the proposed method is call-use displacement. Conservative fail-closed predicates, fixture-level safety checks, the controlled C++ equivalence harness, and the OpenCV source audit provide additional support when deciding whether a candidate can be modeled for displacement. These checks, however, do not

establish semantic preservation for arbitrary C++ transformations and do not replace production-scale pointer, alias, lifetime, or synchronization analysis. The safety claim therefore applies to the limited decision-and-gap-update mechanism investigated in this study rather than to a general automatic source-rewriting system.

The real-project experiment provides useful evidence, but it also exposes an important limitation of the current model. Although the profiling data come from an existing production codebase, the bounded empirical representation captures only a narrow range of that behavior. In addition, the measured parallel program is OpenCV's own threaded implementation rather than an optimized executable generated by DTMC-GA. The measured native speedup of $1.8691\times$ and the modeled speedup of $1.0000\times$ differ by 46.50%. This difference shows that the current model should not be assumed to reproduce native OpenCV parallel performance. Stronger production-runtime claims would require richer dynamic interaction graphs, additional real projects, and direct execution of a DTMC-GA-generated implementation.

The controlled scalability study demonstrates the ability of the sparse implementation to handle graphs with up to 1000 classes under the same search budget. This is a statement about computational feasibility rather than the adequacy of the optimization search. As the system becomes larger, however, a fixed population and generation budget cannot guarantee partitions of comparable quality. Because of this, computational feasibility and search-budget adequacy should be considered as two separate questions.

Table 8 makes this interpretation explicit by mapping each research question to the kind of evidence provided in the manuscript. The table is included to prevent the discussion from sounding broader than the available data.

A key methodological implication is that communication overhead alone is an incomplete basis for judging a partition. A low-communication partition may still be unattractive if load balance is weak or no overlap is available. Conversely, a partition with moderate communication may be useful when remote execution can be hidden behind local computation. Speedup, communication overhead, load balance, and speedup-aware MQ should therefore be interpreted together rather than collapsed into a single structural score.

The reproducibility artifact consists of the retained inputs, random seeds, parameter settings, workloads, and comparison methods, all of which affect the numerical claims. Therefore, results related to modeled speedup, communication overhead, modularization quality, ablation behavior, and sensitivity are regenerated from the released data and scripts. No selection of the best case is performed, and all runs are retained. This makes it possible to examine variation in the stochastic search and the basis of the reported aggregate results.

The main remaining limitations are empirical. The quality of the input model is important to DTMC-GA, and statically estimated call frequencies may differ from behavior observed during execution under real workloads. More complete validation would require dynamic traces, runtime correlation based on measured communication costs and latency, and broader sensitivity studies covering different workloads, latency models, and GA configurations. The current formulation also focuses on class-level modularization of C++ software. Other languages and very large systems may require different parsers, call-resolution approaches, memory-model restrictions, sampling or analysis strategies, incremental analysis, or hierarchical optimization. In its current form, the method is more suitable for offline restructuring and architecture exploration than for unsupervised rewriting of production software without developer review, testing, and rollback support.

Table 8. Research-question-to-evidence mapping for the released fixture-suite study.

RQ	Evidence Used	Bounded Answer in This Manuscript
RQ1	DTMC visit estimates, impact scores, and fixture-level CDG inputs	DTMC weighting exposes execution-relevance information not present in unweighted structural graphs, within the released fixtures.
RQ2	Fitness definition, GA implementation, retained runs, and baseline comparison	GA search can be coupled with DET-based scoring to favor modeled overlap and load balance rather than communication minimization alone.
RQ3	Guarded call-use safety decisions, ablation output, and overlap-window diagnostics	Guarded displacement contributes to the artifact indicators when safety conditions permit, but it remains a conservative opportunity rather than an automatic rewrite.
RQ4	Validation diagnostics for speedup-aware MQ and surrogate modeled speedup	Speedup-aware MQ explains surrogate artifact outputs better than traditional MQ in this benchmark-suite, while still requiring external runtime validation.
RQ5	Descriptive statistics, significance table, and communication-overhead comparison	DTMC-GA shows fixture-bounded speedup and load-balance advantages, while Bunch remains strongest on communication overhead; large-project generalization remains future work.

7. Conclusions

This paper introduces DTMC-GA as a performance-driven approach to modularization of sequential object-oriented software. The method features class-dependence analysis, DTMC-based execution-likelihood modeling, DET-based performance estimation, guarded call-use displacement, and genetic search under communication, load-balance, and deployment constraints. The contribution goes beyond clustering as a single objective by linking the original parallelization problem to a formal optimization model, an executable analysis pipeline, and a reproducible evaluation process.

The main scientific value of the approach is the shift from purely structural modularization toward performance-aware modularization. By considering expected visits, distributive execution time, communication overhead, load balance, and overlap windows together, DTMC-GA provides a formal basis for comparing module boundaries that may support automated parallelization.

The study moves software modularization from a mainly structural perspective toward one that also considers execution frequency, communication cost, load balance, deployment constraints, and conservatively modeled overlap. The fixture-suite experiments remain model-derived rather than measured runtime acceleration and are therefore reported as modeled speedup. Further experiments show that the effect of DTMC visit weighting depends on the workload, that NSGA-III is competitive under matched experimental conditions, and that the sparse implementation remains computationally feasible in the controlled stress series up to 1000 classes. The OpenCV 4.7.0 experiment adds executed instrumentation and a measured native-runtime calibration, but the measured speedup comes from OpenCV's own threaded implementation and not from a source rewrite generated by DTMC-GA. Future validation should include matched DTMC-GA-generated implementations and additional real-world systems.

Author Contributions: Conceptualization, K.A.I. and H.I.; methodology, K.A.I. and H.I.; software, K.A.I.; validation, K.A.I., H.I. and J.K.; formal analysis, K.A.I.; investigation, K.A.I.; resources, H.I. and J.K.; data curation, K.A.I.; writing—original draft preparation, K.A.I.; writing—review and editing, H.I. and J.K.; visualization, K.A.I.; supervision, H.I. and J.K.; project administration, H.I. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The reproducibility artifact, including source code, benchmark fixtures, raw results, statistical scripts, generated figures, Docker configuration, and GitHub Actions CI workflow, is publicly available at <https://github.com/dhok8400-png/DTMC-GA-Reproducibility-Artifact> (accessed on 31 August 2026) and archived on Zenodo with DOI: 10.5281/zenodo.21045304. The core evidence remains the released executable benchmark-suite, supplemented in this revision by a bounded OpenCV 4.7.0 runtime-calibration checkpoint; broader multi-project production validation remains future work. The package includes executable Python 3 modules, retained raw benchmark-suite results, processed statistical outputs, figure-generation scripts, Docker configuration, and the GitHub Actions workflow used for artifact verification.

Conflicts of Interest: The authors declare no conflicts of interest.

References

- Mitchell, B.S.; Mancoridis, S. On the automatic modularization of software systems using the Bunch tool. *IEEE Trans. Softw. Eng.* **2006**, *32*, 193–208. [\[CrossRef\]](#)
- Parsa, S.; Bushehrian, O. The design and implementation of a framework for automatic modularization of software systems. *J. Supercomput.* **2005**, *32*, 71–94. [\[CrossRef\]](#)
- Tabrizi, A.H.F.; Izadkhah, H. Software modularization by combining genetic and hierarchical algorithms. In Proceedings of the 5th Conference on Knowledge Based Engineering and Innovation (KBEI), Tehran, Iran, 28 February–1 March 2019; pp. 454–459. [\[CrossRef\]](#)
- Aghdasifam, M.; Izadkhah, H.; Isazadeh, A. A new metaheuristic-based hierarchical clustering algorithm for software modularization. *Complexity* **2020**, *2020*, 1794947. [\[CrossRef\]](#)
- Pourasghar, B.; Izadkhah, H.; Isazadeh, A.; Lotfi, S. A graph-based clustering algorithm for software systems modularization. *Inf. Softw. Technol.* **2021**, *133*, 106469. [\[CrossRef\]](#)
- Izadkhah, H.; Tajgardan, M. Information theoretic objective function for genetic software clustering. *Proceedings* **2020**, *46*, 18. [\[CrossRef\]](#)
- Teymourian, N.; Izadkhah, H.; Isazadeh, A. A fast clustering algorithm for modularization of large-scale software systems. *IEEE Trans. Softw. Eng.* **2022**, *48*, 1451–1462. [\[CrossRef\]](#)
- Schnoor, H.; Hasselbring, W.; Jung, R. Using genetic algorithms for research software structure optimization. *Software* **2025**, *4*, 26. [\[CrossRef\]](#)
- Arasteh, B.; Seyyedabbasi, A.; Rasheed, J.; Abu-Mahfouz, A.M. Program source-code re-modularization using a discretized and modified sand cat swarm optimization algorithm. *Symmetry* **2023**, *15*, 401. [\[CrossRef\]](#)
- Arasteh, B.; Sefati, S.S.; Kusetogullari, H.; Kiani, F. Generating software architectural model from source code using module clustering. *Symmetry* **2025**, *17*, 1523. [\[CrossRef\]](#)
- Alshareef, H.; Maashi, M. Application of multi-objective hyper-heuristics to solve the multi-objective software module clustering problem. *Appl. Sci.* **2022**, *12*, 5649. [\[CrossRef\]](#)
- Prajapati, A.; Geem, Z.W. Harmony search-based approach for multi-objective software architecture reconstruction. *Mathematics* **2020**, *8*, 1906. [\[CrossRef\]](#)
- Bhosale, A.; Barakshian, P.; Rosas, M.R.; Eigenmann, R. Automatic and interactive program parallelization using the Cetus source to source compiler infrastructure v2.0. *Electronics* **2022**, *11*, 809. [\[CrossRef\]](#)
- Blume, W.; Eigenmann, R.; Faigin, K.; Grout, J.; Hoeflinger, J.; Padua, D.; Petersen, P.; Pottenger, W.; Rauchwerger, L.; Tu, P.; et al. Polaris: Improving the effectiveness of parallelizing compilers. In *Languages and Compilers for Parallel Computing*; Springer: Berlin/Heidelberg, Germany, 1994; pp. 141–154. [\[CrossRef\]](#)
- Hall, M.W.; Anderson, J.M.; Amarasinghe, S.P.; Murphy, B.R.; Liao, S.-W.; Bugnion, E.; Lam, M.S. Maximizing multiprocessor performance with the SUIF compiler. *Computer* **1996**, *29*, 84–89. [\[CrossRef\]](#)
- Grosser, T.; Groesslinger, A.; Lengauer, C. Polly—Performing polyhedral optimizations on a low-level intermediate representation. *Parallel Process. Lett.* **2012**, *22*, 1250010. [\[CrossRef\]](#)
- Acun, B.; Gupta, A.; Jain, N.; Langer, A.; Menon, H.; Mikida, E.; Ni, X.; Robson, M.; Sun, Y.; Totoni, E.; et al. Parallel programming with migratable objects: Charm++ in practice. In Proceedings of the SC14: International Conference for High Performance Computing, Networking, Storage and Analysis, New Orleans, LA, USA, 16–21 November 2014; pp. 647–658. [\[CrossRef\]](#)
- Tilevich, E.; Smaragdakis, Y. J-Orchestra: Enhancing Java programs with distribution capabilities. *ACM Trans. Softw. Eng. Methodol.* **2009**, *19*, 1. [\[CrossRef\]](#)
- Spiegel, A. Automatic Distribution of Object-Oriented Programs. Ph.D. Thesis, Freie Universität Berlin, Berlin, Germany, 2003. Available online: <https://refubium.fu-berlin.de/handle/fub188/10755> (accessed on 15 July 2026).

20. West, S.; Nanz, S.; Meyer, B. Efficient and reasonable object-oriented concurrency. In Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE 2015), Bergamo, Italy, 30 August–4 September 2015; pp. 734–744. [\[CrossRef\]](#)
21. Bauer, M.; Treichler, S.; Slaughter, E.; Aiken, A. Legion: Expressing locality and independence with logical regions. In Proceedings of the SC12: International Conference for High Performance Computing, Networking, Storage and Analysis, Salt Lake City, UT, USA, 10–16 November 2012; pp. 1–11. [\[CrossRef\]](#)
22. Bosilca, G.; Bouteiller, A.; Danalis, A.; Faverge, M.; Herault, T.; Dongarra, J.J. PaRSEC: Exploiting heterogeneity to enhance scalability. *Comput. Sci. Eng.* **2013**, *15*, 36–45. [\[CrossRef\]](#)
23. Nickolls, J.; Buck, I.; Garland, M.; Skadron, K. Scalable parallel programming with CUDA. *Queue* **2008**, *6*, 40–53. [\[CrossRef\]](#)
24. Dagum, L.; Menon, R. OpenMP: An industry standard API for shared-memory programming. *IEEE Comput. Sci. Eng.* **1998**, *5*, 46–55. [\[CrossRef\]](#)
25. Li, Q.; Lu, M.; Gu, T.; Wu, Y. Runtime software architecture-based reliability prediction for self-adaptive systems. *Symmetry* **2022**, *14*, 589. [\[CrossRef\]](#)
26. Cheung, R.C. A user-oriented software reliability model. *IEEE Trans. Softw. Eng.* **1980**, *SE-6*, 118–125. [\[CrossRef\]](#)
27. Gokhale, S.S.; Wong, W.E.; Horgan, J.R.; Trivedi, K.S. An analytical approach to architecture-based software performance and reliability prediction. *Perform. Eval.* **2004**, *58*, 391–412. [\[CrossRef\]](#)
28. Souravlas, S.; Anastasiadou, S.D.; Tantalaki, N.; Katsavounis, S. A fair, dynamic load balanced task distribution strategy for heterogeneous cloud platforms based on Markov process modeling. *IEEE Access* **2022**, *10*, 26149–26162. [\[CrossRef\]](#)
29. Harman, M.; Jones, B.F. Search-based software engineering. *Inf. Softw. Technol.* **2001**, *43*, 833–839. [\[CrossRef\]](#)
30. Harman, M.; Mansouri, S.A.; Zhang, Y. Search-based software engineering: Trends, techniques and applications. *ACM Comput. Surv.* **2012**, *45*, 11. [\[CrossRef\]](#)
31. Praditwong, K.; Harman, M.; Yao, X. Software module clustering as a multi-objective search problem. *IEEE Trans. Softw. Eng.* **2011**, *37*, 264–282. [\[CrossRef\]](#)
32. Deb, K.; Pratap, A.; Agarwal, S.; Meyarivan, T. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evol. Comput.* **2002**, *6*, 182–197. [\[CrossRef\]](#)
33. Tan, A.J.J.; Chong, C.Y.; Aleti, A. E-SC4R: Explaining software clustering for remodularisation. *J. Syst. Softw.* **2022**, *186*, 111162. [\[CrossRef\]](#)
34. Meng, F.; Wang, Y.; Chong, C.Y.; Yu, H.; Zhu, Z. CLUE: Customizing clustering techniques using machine learning for software modularization. In Proceedings of the 15th Asia-Pacific Symposium on Internetware (Internetware 2024), Macau, China, 24–26 July 2024; pp. 189–198. [\[CrossRef\]](#)
35. Isazadeh, A.; Karimpour, J.; Elgedawy, I.; Izadkhah, H. An analytical model for source code distributability verification. *J. Zhejiang Univ. Sci. C* **2014**, *15*, 126–138. [\[CrossRef\]](#)
36. Lattner, C.; Adve, V. LLVM: A compilation framework for lifelong program analysis and transformation. In Proceedings of the International Symposium on Code Generation and Optimization (CGO), Palo Alto, CA, USA, 20–24 March 2004; pp. 75–86. [\[CrossRef\]](#)
37. Dean, J.; Grove, D.; Chambers, C. Optimization of object-oriented programs using static class hierarchy analysis. In *ECOOP '95—Object-Oriented Programming*; Springer: Berlin/Heidelberg, Germany, 1995; pp. 77–101. [\[CrossRef\]](#) [\[PubMed\]](#)
38. Ferdinand, C.; Heckmann, R. AiT: Worst-case execution time prediction by static program analysis. In *Building the Information Society*; Springer: Boston, MA, USA, 2004; pp. 377–383. [\[CrossRef\]](#)
39. Tidorum Ltd. Bound-T Time and Stack Analyser. Available online: <https://www.bound-t.com/> (accessed on 15 July 2026).
40. Ballabriga, C.; Cassé, H.; Rochange, C.; Sainrat, P. OTAWA: An open toolbox for adaptive WCET analysis. In *Software Technologies for Embedded and Ubiquitous Systems*; Springer: Berlin/Heidelberg, Germany, 2010; Volume 6399, pp. 35–46. [\[CrossRef\]](#)
41. Li, X.; Liang, Y.; Mitra, T.; Roychoudhury, A. Chronos: A timing analyzer for embedded software. *Sci. Comput. Program.* **2007**, *69*, 56–67. [\[CrossRef\]](#)
42. Rapita Systems Ltd. RapiTime: Worst-Case Execution Time Analysis. Available online: <https://www.rapitasystems.com/products/rapitime> (accessed on 15 July 2026).
43. Garey, M.R.; Johnson, D.S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*; W. H. Freeman: San Francisco, CA, USA, 1979. Available online: <https://dblp.org/rec/books/fm/GareyJ79> (accessed on 15 July 2026).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.